

Abstract

Search and rescue operations now depend on drones because these air vehicles enable safe access to dangerous zones while delivering immediate situation awareness. The deployment of these drones in complex urban areas needs advanced path planning systems which optimize routes while responding to changing environmental conditions. The A* algorithm and reinforcement learning are widely used in path planning tasks but the A* algorithm fails to operate effectively in changing environments, yet pure reinforcement learning methods need extended time to learn and consume many samples. The research develops a combined path planning approach which uses A* and MR-Qlearning to achieve both global optimization and immediate local adjustments in unpredictable environments. The system uses A* to generate the best initial path before MR-Qlearning with reward optimization and heuristic-based exploration takes over for dynamic obstacle prevention. The combination approach operated successfully through a custom 2D grid-based simulation environment built with Python to navigate environments containing stationary and moving obstacles. The path optimization method achieved better success rates than reinforcement learning alone while producing smaller path deviations. The system establishes a functional system for autonomous search and rescue operations which achieves both fast processing and flexible operation.

Keywords: Drone path planning, A* algorithm, Reinforcement Learning (RL), dynamic obstacle Avoidance, unmanned aerial vehicle (UAV),

Contents

Chapter 1 Topic Introduction.....	1
1.1 Source.....	1
1.2 Background	1
1.3 Significancy.....	3
1.4 Key Research Problems	4
Chapter 2 Related Work.....	7
2.1 Machine Learning Based Algorithm Systems.....	7
2.2 A-Star Algorithm for Drone System	14
2.3 Search and Detection Operation in drone system	16
Chapter 3 Methodology	18
3.1 Overall Workflow	19
3.2 Static Global Path Planner Based on A* Algorithm.....	20
3.3 Adaptive Navigator Via MR-Qlearning (Modified Reward)	23
3.4 Path Optimization by Combining A* and MR-QLearning Approach.....	27
Chapter 4 System and Environment Implementation	33
4.1 Development and Demonstration Environment.....	33
4.2 Testing and Evaluation Environment	35
Chapter 5 Experiment and Results.....	40
5.1 Performance Metrics	40
5.2 Experimental Results and Comparative Analysis	43
Chapter 6 Conclusion and Future Work.....	49
6.1 Limitations and Constraints	49
6.2 Future Research Directions	50
6.3 Conclusion.....	50
References	52
Acknowledgment	56

插图清单

Figure 1 Concept of Drone Location and Id Information Sharing System	8
Figure 2 Methodology Overview	18
Figure 3 Research Components Diagram	18
Figure 4 A* Algorithm.....	20
Figure 5 Reinforcement Learning Algorithm	23
Figure 6 Flowchart Diagram of The Reward Function	24
Figure 7 Combination Approach.....	27
Figure 8 Flowchart Diagram of the A* Fallback Mechanism	30
Figure 9 Path Optimization Approach Flowchart	32
Figure 10 Environment Setup	33
Figure 11 Development and Demonstration Grid Environment	34
Figure 12 Testing and Evaluation Environment	35
Figure 13 workspace including start, target, obstacle from the top view, as well as the actions [25]	38

附表清单

Table 1 Approaches Comparison..... 7

Table 2 Testing Parameters 40

Table 3 Experimental Results Across 10 Environment Configurations 43

缩略语说明

A*: A-Star search algorithm

GUI: Graphic User Interface

ML: Machine Learning

RL: Reinforcement Learning

MR-QLearning: Modified Reward Q-Learning

AI: Artificial Intelligence

HRL: Hierarchical Reinforcement Learning

IOT: Internet of Things

UAV: Unmanned Aerial Vehicle (Drone)

USAR: Urban Search and Rescue

Chapter 1 Topic Introduction

1.1 Source

This research implements a comprehensive approach to drone path planning and optimization within a simulated environment, leveraging a combination of the A* algorithm and reinforcement learning (RL). The primary objective is to develop a robust framework allowing the drone to autonomously navigate complex terrains with static and dynamic obstacles to reach its target efficiently in urban search and rescue missions. The environment is simulated as a 2D grid using Python, where obstacles, targets, and environmental dynamics are defined. The A* algorithm provides an initial optimal path for the drone based on static obstacles, ensuring an efficient and collision-free trajectory. To enhance adaptability in dynamic environments, reinforcement-learning is integrated into the system utilized to train the drone to adjust its path in real time, optimizing its movements based on a reward system that encourages target proximity and penalizes collisions or inefficient routes. This research lays a foundation for developing intelligent autonomous systems for search and rescue, delivery services, and other real-world applications.

1.2 Background

Drones have become transformative tools in various civilian and emergency response applications. Their capabilities enormously enhance the efficiency of operations, demonstrating their potential to contribute positively to society. These autonomous or remotely piloted aircraft systems offer mobility, flexibility, and aerial perspective capabilities, making them particularly valuable in time-critical scenarios such as search and rescue (SAR) operations. The integration of drones in SAR operations has shown remarkable potential to enhance the efficiency and effectiveness of emergency response efforts, especially in scenarios where traditional ground-based search methods face significant limitations or risks that the drones cannot be affected by.

Search and rescue missions are challenging and time-sensitive missions aimed at finding and rescuing people in distress or imminent danger. The conventional approach to SAR operations, which primarily relies on ground teams and human-crewed aircraft, often faces challenges such as limited visibility, difficult terrain accessibility, and significant resource requirements regarding personnel and equipment.

Through their unique characteristics, drones present a compelling solution to many challenges. Their ability to rapidly deploy, operate in challenging environments, provide real-time aerial

surveillance, and operate in a timely manner makes them invaluable assets in SAR missions. Moreover, the reasonably low operational costs and reduced risk to human personnel make drones attractive for emergency response organizations. However, effectively utilizing drones in SAR operations requires educated path planning and optimization capabilities to maximize their potential impact.

Path planning, in the context of drone operations, refers to locating and determining the optimal route for a drone to navigate through from its current position to a target location while considering various constraints and objectives. These constraints typically include obstacle avoidance, battery limitations, and environmental conditions which affect the mission's success. In SAR scenarios, effective path planning becomes particularly crucial as it directly impacts the probability of successfully locating the operations targets in the shortest possible time while managing limited resources efficiently.

The complexity of drone path planning and optimization in SAR missions is heightened by several factors:

1. Real time Decision making: The need to process and respond to dynamic environmental information and changing mission parameters in real-time
2. resource optimization and balancing: managing limited battery life while maximizing search coverage
3. Environmental challenges: navigating through complex urban or natural environments with various obstacles.
4. target uncertainty: operating with incomplete or uncertain information about the target precise location.
5. time Criticality: balancing thorough search patterns with the urgency of the mission

Traditional path-planning approaches often struggle to address these challenges systematically, particularly in complex urban environments with numerous obstacles and highly constrained search space. While existing research has made significant progress in individual aspects of drone navigation, there remains a crucial need for integrated solutions that can handle real-time path planning while optimizing it.

Recent achievements in artificial intelligence and autonomous systems have opened new possibilities for enhancing drone capabilities in SAR operations. Particularly, developments in

areas such as reinforcement learning and computer vision have shown promising results in improving autonomy and effectiveness of drone operations. However, the integration of these technologies and methods into a cohesive system capable of real-time operation in complex environments remains a significant challenge.

This research addresses these challenges by developing an intelligent real-time path planning and optimization system designed explicitly for drone-based SAR missions.

The proposed system aims to improve SAR operations Through:

- Real-time path planning with dynamic obstacle avoidance capabilities.
- battery-aware routing algorithms that maximize search coverage in the targeted area.
- adaptive navigation strategies for complex urban environments.

1.3 Significancy

The military and civilian sectors now depend on drones because these systems provide flexible operation and multiple operational benefits. The military uses drones to protect personnel from dangerous situations during reconnaissance and surveillance operations [1],[2] while achieving better cost performance than manned aircraft systems [3],[4]. The unmanned systems operate in dangerous zones while maintaining long surveillance periods and accessing areas which human operators cannot reach [4] to deliver tactical benefits through immediate intelligence collection and safe operations in hostile areas [4].

The use of drones has transformed various industries through their ability to perform fast damage assessments and supply emergency aid during natural disasters [4],[2]. The combination of safety and operational efficiency enables drone systems to inspect bridges and power lines and tall structures while they revolutionize agricultural practices through precise crop tracking. Emergency services now depend on drone technology to perform firefighting duties and law enforcement work and search and rescue operations while environmental protection benefits from improved wildlife tracking and environmental assessment tools [2]. Drone technology serves multiple domains through its applications in urban planning which include aerial mapping and urban development monitoring [2],[5].

The essential role of drones in search and rescue operations becomes evident through multiple essential elements. The system provides fast coverage of extensive areas which ground teams need extended periods to search [1],[2] and enables access to dangerous and distant locations that ground teams cannot reach [4] and provides aerial views that help detect hidden patterns and clues [5]. The

fast deployment of drones enables quick response times because they can arrive at search sites within short periods [3],[5] while providing instant visual data transmission to rescue teams [2],[4] and maintaining continuous search operations under suitable weather conditions [4],[3]. The system provides resource optimization through its affordable operations which outperform manned aircraft [3] and enables better ground team deployment through drone observations [3],[4] and protects personnel from dangerous situations [2]. The search process becomes more effective because drones can use multiple sensors to detect different search environments [5] while following predetermined search patterns to maximize coverage [3] and automatically recording searched areas and discovered evidence for operational coordination and documentation purposes [2].

1.4 Key Research Problems

The research solves several critical issues in autonomous drone path planning for search and rescue missions through the implementation of a Path optimization A* and reinforcement learning system.

1. Dynamic Obstacle Navigation in Real-Time

The research solves the essential problem of navigating through changing obstacles by uniting traditional planning methods with machine learning-based adaptation techniques. The main drawback of A* path planning algorithms and similar methods is their inability to handle dynamic environments because they were designed for static conditions which do not match the typical search and rescue operations. The system requires immediate replanning functions which A* does not support because of its static environment design. The system uses an advanced tracking system to always check the execution plan. The system triggers its advanced MR-QLearning algorithm for local path adjustments when it detects moving obstacles that block the planned route. The system makes instant route changes without needing to rebuild the entire path which allows it to continue mission advancement while steering clear of new obstacles. The system uses local adaptation to find detours for immediate obstacles while keeping the global A* plan's strategic path.

2. The Need for Optimality and Adaptability

The research establishes a two-layered framework to address the core conflict between finding the best path and environmental flexibility. The A* algorithm generates optimal routes in fixed environments but fails to adapt when moving obstacles block its pre-determined path. The pure reinforcement learning methods provide flexibility, but they fail to achieve optimal results and need extended training time to converge. The hybrid system uses A* to find the most efficient route through static obstacles during initial planning but uses an improved MR-QLearning with heuristic-

based exploration to adapt to dynamic obstacles. The system combines proven optimal methods with learning-based adaptation to create a complete solution for search and rescue operations that face unpredictable environmental changes.

3. Inefficiency in Pure Reinforcement Learning approach

The research solves the major problem of sample inefficiency in pure reinforcement learning methods by strategically uniting them with traditional learning approaches. The standard Q-learning method needs long exploration periods and multiple training sessions to create effective policies which make it unsuitable for time-sensitive search and rescue missions that need fast deployment. The solution implements three efficiency improvements which include an enhanced MR-QLearning algorithm that uses heuristic-based exploration to direct action choices toward goal advancement and minimizes aimless movement and a combination system that employs A* for both global path generation and backup planning to reduce the search area for reinforcement learning and local training areas that concentrate learning on essential sections of the environment. The combination of new approaches allows successful operation with minimal training time which makes this solution suitable for real-world implementation.

4. Battery Efficiency and Energy-Aware Path Planning

The research solves battery efficiency problems by creating an integrated system which adds energy monitoring to route optimization algorithms. The high computational expenses of complex path planning systems combined with their lack of battery constraint consideration led to mission failure when drones run out of power before finishing their tasks. The search and rescue operations require drones to finish their assignments while maintaining enough power for secure flight and return to base. The solution uses a proportional energy consumption model to monitor battery depletion based on path distance which creates authentic energy limitations during simulation. The MR-QLearning component uses step penalties in its reward function to indirectly support energy savings through path length reduction. The A* algorithm provides global optimization for route selection to reduce unnecessary detours while the local adaptation features stop drones from wasting energy by trying to pass through blocked areas multiple times.

5. Robustness to Environmental Uncertainty

The research implements multiple reliability assurance layers to enhance system resistance against environmental uncertainties. Search and rescue operations function with limited environmental data so systems need to maintain stability through uncertain conditions and limited visibility

without experiencing critical breakdowns. The solution includes a complete fallback system which maintains mission success when the learning component faces situations that exceed its trained experience. The reliability layer implements A* algorithm global replanning when the MR-QLearning agent fails to discover a workable path because of computational restrictions. The system operates with multiple recovery protocols which perform local adjustments for small obstacles and full system replanning for major environmental transformations. The system achieves better mission success rates through its layered robustness design which handles various operational environments from controlled static spaces to unpredictable dynamic search and rescue situations.

Chapter 2 Related Work

Table 1 Approaches Comparison

Approach	How It Works	Advantages	Disadvantages
A Algorithm*	Uses a heuristic function to calculate the shortest path from the starting point to the goal based on the cost function.	- Efficient for static environments	- Poor performance in dynamic environments
Dijkstra's Algorithm	Explores all possible paths and selects the shortest one based on cost.	- Guarantees shortest path	- Computationally expensive
		- Suitable for simple environments	- Slow for large environments
Rapidly-exploring Random Tree (RRT)	Expands random paths towards the target and refines them as needed.	- Fast for pathfinding in complex environments	- Inefficient in narrow spaces
		- Suitable for non-uniform maps	- May not always find the shortest path
Potential Field Method	Treats the goal as an attracting force and obstacles as repelling forces, generating a potential field for movement.	- Easy to implement	- Stuck in local minima
My Approach (Reinforcement Learning)	Trains a model using trial-and-error (reward-based) learning to adapt and find the optimal path based on real-time environmental feedback.	- Adaptable to dynamic environments	- Requires extensive training data

2.1 Machine Learning Based Algorithm Systems

2.1.1 Real time 3D Path Planning for UAV Navigation

Tullu et al. [6] proposed a machine-learning approach to real-time 3D path planning for autonomous UAV navigation in cluttered environments. The authors approach involved integrating YOLO (You Only Look Once) object detection algorithm with a lightweight 3D path planner to enable real-time obstacle avoidance. Their approach differs from traditional path planning algorithms like A* by avoiding exhaustive node exploration and instead mapping the UAV's current location to select nodes between consecutive obstacles. The system uses a ZED mini stereo camera for object detection and localization, combined with GPS and LiDAR for UAV positioning. The path planner considers both open obstacles (requiring passage through, like windows) and closed obstacles (requiring avoidance maneuvers), applying a heuristic function for cost minimization. The effectiveness of their approach was through Software-In-The-Loop (SITL) simulations tests in Gazebo environments and preliminary real-world flight tests. Results demonstrated successful

autonomous navigation through various obstacles, including pedestrians, windows, poles, tunnels, and trees. An essential contribution is decreasing the computational burden on the companion computer by integrating fast object detection with efficient path planning, making it suitable for real-time applications. However, limitations included GPS accuracy issues in open fields and the need for further real-world testing in more complex environments.

2.1.2 Data Analysis and Modeling for Drone Communications using ML algorithms

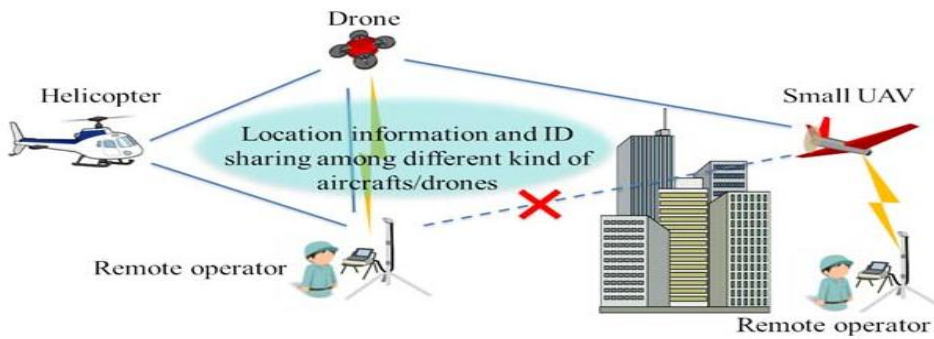


Figure 1 Concept of Drone Location and Id Information Sharing System

Shan, R. Miura, T. Kagawa,[7] presented a research paper on unmanned aerial vehicle (UAV), also called a drone, is getting increasingly important in many emerging technology areas. For communication area, the drone also takes an important role in lots of significant topics like emergency communications, device-to-device (D2D) communications, and the Internet of Things (IoT). One of the important drone applications is to collect and share data among drones and other aircraft, which is useful for drone control so that dangerous conditions can be avoided. In particular, the drone control and safety guarantees are difficult to attain, especially, when drones fly beyond the line of sight (BLOS). For this reason, we develop a drone location information sharing system using the 920-MHz band. We use this system to do a long distance propagation field experiment for model establishment. Unfortunately, the current data collection for model establishment work needs a great effort and time to do experiments to collect a huge number of data for data analysis so that a suitable model can be established. Therefore, in this paper, the authors propose a novel method, which is based on machine learning approach, to data analysis and model establishment for drone communications, so that the effort and cost for establishing model can be reduced and a model, which captures more details about the drone communications, can be obtained. The results

of this paper validate that the proposed method can indeed establish a more complicated model with less effort. Specifically, from the distribution of the training error, it can be known that there are over 80% training errors with intensity less than 5, which ensures the error performance of the proposed method.

2.1.3 Advanced implementation in drone for Smart City Developments using ML Algorithm

Sampath Boopathi [8] presented a chapter that explores the integration of drones, machine learning, and artificial intelligence (AI) in smart city development. Drones can revolutionize urban planning, energy efficiency, noise reduction, environmental monitoring, traffic management, infrastructure inspection, public safety, data security, and privacy protection. AI-driven solutions enable data-driven decision-making for resource allocation, sustainability, and predictive modeling. AI optimizes flight paths for energy efficiency, noise reduction strategies enhance drone social acceptance, and autonomous drone navigation is crucial for safe urban deployment. Drones optimize traffic flow, reduce congestion, and enhance safety. AI provides predictive insights into traffic patterns, and drones aid in law enforcement, emergency response, and first responder support. Data security and privacy protection measures are essential for maintaining public trust.

2.1.4 Sound-Based Amateur Drone Detection for Public Safety Applications

Muhammad Zohaib Anwar,[9] presented that In recent years, popularity of unmanned air vehicles enormously increased due to their autonomous moving capability and applications in various domains. This also results in some serious security threats, that needs proper investigation and timely detection of the amateur drones (ADr) to protect the security sensitive institutions. In this paper, the author propose the novel machine learning (ML) framework for detection and classification of ADr sounds out of the various sounds like bird, airplanes, and thunderstorm in the noisy environment. To extract the necessary features from ADr sound, Mel frequency cepstral coefficients (MFCC), and linear predictive cepstral coefficients (LPCC) feature extraction techniques are implemented. After feature extraction, support vector machines (SVM) with various kernels are adopted to accurately classify these sounds. The experimental results verify that SVM cubic kernel with MFCC outperform LPCC method by achieving around 96.7% accuracy for ADr detection. Moreover, the results verified that the proposed ML scheme has more than 17% detection accuracy, compared with correlation-based drone sound detection scheme that ignores ML prediction.

2.1.5 Reinforcement learning framework for UAV SAR For optimization of path

Ramezani and Amiri Atashgah [10] displayed an energy-aware hierarchical reinforcement learning (HRL) framework for unmanned air vehicle (UAV) search and rescue missions that integrate LSTM-based battery consumption prediction for optimizing path planning. Their framework involves a high-level controller (HLC) that sets feasible goals based on predicted battery requirements and a low-level controller (LLC) that executes navigation commands. A novel innovation is using a bidirectional LSTM model to accurately predict battery consumption for different targets, helping the HLC make energy-efficient decisions. The framework includes hindsight experience replay (HER) to improve learning efficiency and adaptability. The system was validated through extensive simulations in MATLAB using four test environments with varying obstacle densities and target counts. Results demonstrated superior performance compared to baseline methods, achieving 90-95% success rates across environments while maintaining low collision rates of 3-8%. The authors' hierarchical approach with LSTM battery prediction enabled more efficient path planning and energy utilization compared to traditional reinforcement learning methods. Key advantages included setting achievable goals based on energy constraints and adapting to dynamic environments while optimizing battery usage through predictive insights. Test results verified the framework's effectiveness for autonomous UAV navigation in complex search and rescue scenarios.

2.1.6 Deep learning approach in a multi-obstacle environment

In this study, Zhang et al. [14] displayed a deep reinforcement learning (DRL) method for autonomous UAV navigation in dynamic multi-obstacle environments. The key methodological contributions include 1) the Development of a two-stream Actor-Critic network structure based on Twin Delayed Deep Deterministic Policy Gradients (TD3) that considers both current observations and changes in environment observations to predict obstacle movements, 2) the Implementation of a Unity3D simulation platform to reduce the reality gap, and 3) Design of a comprehensive reward function incorporating distance, step penalty, angle penalty and obstacle avoidance components. The method was evaluated through extensive experiments in both static and dynamic environments with varying obstacle configurations. Results demonstrated superior performance compared to baseline methods:

- In static environments with random obstacles, their method achieved up to 95.25% success rate compared to 92.10% for DDPG and 95.15% for conventional TD3
- In dynamic environments with moving obstacles, their approach maintained a 77.65% success rate even at high obstacle speeds (20 m/s), while DDPG dropped to 59.80%
- The method showed better generalization ability and faster convergence during training compared to baselines.
- The two-stream network structure enabled effective prediction of obstacle movements, allowing proactive collision avoidance.
- Ablation studies on reward function design validated their formulation's ability to balance safety and efficiency

2.1.7 Air-to-Air Simulated Drone Dataset for AI-powered problems

The Air-to-Air Simulated Drone Dataset (A2A-SDD) [12] significantly contributes to counter-UAV research through its comprehensive simulation-based approach to data generation [12]. Using the AirSim platform, this dataset provides diverse drone interaction scenarios, featuring up to three monitoring drones pursuing one or two target drones in various environmental conditions [12]. The dataset's architecture incorporates five different drone types, including several DJI models (Phantom, Mavic, FPV, and Inspire) alongside a generic quadrotor model, offering a broad representation of common UAV platforms [12]. A key innovation of A2A-SDD lies in its multi-view data collection methodology [12]. The dataset provides richer spatial information by capturing scenarios from multiple camera perspectives than traditional single-view approaches. This multi-perspective design is complemented by comprehensive annotations, including precise XYZ coordinates for all drones, detailed bounding box information, and payload status indicators [12]. The dataset also incorporates various environmental conditions, simulating sunny, rainy, and snowy weather across different terrains, from natural landscapes to urban environments [12]. The practical utility of A2A-SDD has been demonstrated through experimental validation using deep learning models [12]. The researchers achieved notable results in two critical areas: distance estimation and payload detection. The distance estimation models achieved an RMSE of approximately 8.4 meters when tested on real-world data, while payload detection systems demonstrated accuracies between 87-92% in practical applications [12]. These results validate the dataset's effectiveness in bridging the gap between simulated training data and real-world

applications. However, A2A-SDD does face certain limitations [12]. The reliance on simulation, while enabling comprehensive data collection and annotation, raises questions about fully representing real-world complexities. The current focus on visual data also allows expansion into other sensing modalities [12]. Despite these limitations, the dataset is valuable for developing and testing counter-UAV measures, particularly in scenarios where real-world data collection would be impractical or cost-prohibitive. The development of A2A-SDD marks a significant step forward in counter-UAV research, providing researchers with a robust platform for developing and testing detection, tracking, and classification algorithms [12]. The dataset's comprehensive nature, combined with its validated effectiveness in real-world applications, establishes it as a valuable resource for advancing the field of drone detection and counter-measure development.

2.1.8 A mixed deep learning approach for planning UAV routes in changing conditions

Zhang [23] presented a paper with the use of Unmanned Aerial Vehicles (UAVs) in military operations has grown in importance. UAVs are typically operated in uncertain environments during actual missions, necessitating careful planning to adjust in real time. For UAVs to navigate through challenging terrain, an efficient path-planning algorithm is also necessary. But for UAV path planning, the widely utilized Rapidly-exploring Random Tree (RRT) method frequently produces less-than-ideal routes that need a lot of post-processing to make them better. Its variations have higher memory and compute costs, but they can converge to the best answer. However, purely learning-based approaches could lead to poor planning or overly lengthy courses created in order to avoid obstacles. In this study, the authors provide a neural network architecture including CNN, LSTM, and an attention mechanism for UAV path planning that combines a variety of sensor inputs. The neural module and informed sampling techniques are then combined by the deep learning-enhanced RRT algorithm to provide adaptive and effective pathfinding in dynamic contexts. Our method outperforms the state-of-the-art Layered RQN, Informed RRT*, and Batch Informed Trees (BIT*) algorithms in terms of overall performance after carefully weighing the trade-offs between path optimality, computing efficiency, and fewer turning angles.

2.1.9 Using multi-agent reinforcement learning for drone mapping

Riccardo Zanol [26] presented a paper explaining drone mapping and surveying for objects of interest, such forest fires, landslides, or wild animals, has become more popular in recent years across a variety of academic communities. However, the usefulness of the drones is significantly

limited by the requirement for a human pilot or a pre-planned flight path, particularly when an entire swarm is deployed. In order to demonstrate that the performance loss resulting from the absence of explicit optimization and pre-programmed knowledge of the system statistics is minimal in the swarm scenario, we present a model of the drone survey problem and apply three well-known reinforcement learning techniques.

2.1.10 Dynamic Reward-Enhanced Q-Learning for Mobile Robot Path Planning

Gharbi [27] developed Dynamic Reward-enhanced Q-learning (DRQL) as a new method to guide mobile robots through unknown spaces while avoiding obstacles. The research combines Q-learning methods with adaptive reward systems to enhance the speed of convergence and exploration performance in standard reinforcement learning systems. The DRQL system uses an advanced reward system which evaluates both target proximity and obstacle distance to deliver advanced feedback to the learning system beyond basic reward systems. The system uses a reward calculation system which adjusts its output based on the robot's current position relative to its target and its distance from surrounding obstacles. The reward system gives positive rewards to the agent when it approaches the target but it penalizes the agent when it moves away from the target or gets too close to obstacles. The dynamic reward system helps the agent learn better paths because it minimizes random exploration and leads to faster policy discovery. The system consists of three operational stages which start with exploration followed by combined exploration and exploitation before ending with final exploitation to solve complex path planning problems. The research team performed extensive simulations using three different environment setups which included different obstacle densities. The results showed that DRQL performed better than SRQL in all evaluation metrics because it reduced movement steps by 9.23-20.78% while maintaining superior reward accumulation during learning. The dynamic reward system enabled the robot to discover paths more efficiently because it achieved goal position faster in all tested environments. The research develops an improved learning system for mobile robots which uses reinforcement learning adaptability to benefit from dynamic reward-based strategic guidance. The research conducted its tests within simulated environments yet real-world testing remains essential to determine the system's practical implementation in physical robotic systems. The DRQL method demonstrates excellent potential for navigating through uncharted and partially known areas because it handles complex path planning problems effectively.

2.2A-Star Algorithm for Drone System

2.2.1 UAV trajectory optimization using the A-star algorithm and clustering-based strategic 3D deployment for improved disaster response

Humairah Hamid [13] presented a broad spectrum of communication and information technologies is currently being investigated for their potential applications in disaster management. A high level of situational awareness, combined with a prompt and accurate response, is essential for the preservation of life during catastrophe scenarios. This study presents a novel communication strategy employing Unmanned Aerial Vehicles (UAVs) as aerial base stations for providing connectivity to the affected area. The system takes advantage of the flexibility and quick deployment characteristics of UAVs. The main focus is to determine the optimal UAV deployment along with trajectory planning to ensure connectivity in areas where conventional base stations are inaccessible. The proposed system employs two types of UAVs: cluster UAVs which act as stationary base stations and relay UAVs acting as mobile base stations. A three-step strategy is proposed to find the suitable location of cluster UAVs, optimize their height and power, and find the optimal trajectory of the relay UAVs to maximize the percentage of users served. Gaussian Mixture Model (GMM) clustering is employed to determine the optimal horizontal location of cluster UAVs. An optimization problem is framed for finding out the optimal height and power for cluster UAVs. Heuristic-based A-star algorithm is used to find out the trajectory of the relay UAVs which can efficiently minimize the overall path length while avoiding obstacles. The simulation results confirm the effectiveness of the proposed approach and demonstrate performance enhancement by comparing it with the benchmark schemes.

2.2.2 Improved method using A-Star algorithm for model R5DOS

Based on the R5DOS model,[15] this paper suggests a three-dimensional space UAV path planning model to address the issues of the A-star algorithm's lengthy computation times and massive computation volumes in three-dimensional space. Together, the enhanced A-star algorithm and the improved R5DOS intersection model are used. They build a local search process together, and by lowering the number of search nodes, the R5DOS path planning model is established. By simulating the path planning model with MATLAB software, the model can significantly lower the computational complexity and number of nodes of the A-star algorithm in three-dimensional spaces, as well as the UAV's calculation time. Lastly, the authors contrast the

refinement of the A-star algorithm with both the original and geometric versions. It is demonstrated by the final fitting result that the enhanced A-star algorithm requires fewer node visits and has a shorter computation time. In general, the simulation results validate the efficacy of the enhanced A-star algorithm and can serve as a guide for further path planning algorithm research.

2.2.3 Optimal route planning for stealth unmanned aerial vehicles in a 3D network radar environment using a modified version of the A-Star algorithm

Zhe Zhang [11], presenter a paper on Path security and search efficiency of penetration paths are the two most crucial aspects of mission execution for stealth unmanned aerial vehicles (UAVs). The optimal penetration path planning technique that takes into account the network radar system, the dynamic radar cross-section of stealth UAVs, and kinematics principles is examined in this article. The traditional A-Star algorithm was modified to include a 3D bidirectional sector multilayer variable step search strategy and the radar threat estimation function. This modified A-Star algorithm seeks to meet both the algorithm searching efficiency and waypoint accuracy requirements. Attitude angle constraints and the radar system's rank-K fusion criterion were then met by the simultaneous selection of new waypoints using the suggested penetration path planning method. The conventional A-Star algorithm, bidirectional multilayer A-Star algorithm, and modified A-Star algorithm were also used for comparative analysis of various algorithms in order to resolve the penetration path issue that UAVs encounter under different threat scenarios. In a 3D complex network radar environment, the simulation results demonstrate the efficacy of the suggested path planning scheme by showing that the paths produced by using the modified algorithm have optimal path costs and higher safety.

2.2.4 The Implementation of A* Algorithm for Global Path Planning in Autonomous

The research by Arfinati et al. [33] demonstrated A* algorithm implementation for autonomous ground vehicle path planning through complex road environments with moving obstacles. The researchers used a kinematic vehicle model in their MATLAB simulation to test the algorithm on an 800-meter curved road with four moving obstacle vehicles. The A* algorithm operated within a binary occupancy grid mapping system which used 0 to indicate free spaces and 1 to show occupied areas. The authors obtained an average total estimated cost value of 2475 while the vehicle moved 139 meters during 10 seconds at a fixed speed of 15 m/s. The research proved A*'s ability to create

optimal paths which both minimize distance and ensure safety in changing environments, but the study focused on ground vehicle kinematics and fixed road layouts instead of drone-specific aerial navigation flexibility.

2.3 Search and Detection Operation in drone system

2.3.1 Path planning algorithm based on Bayesian Inference

M. Khosravi et al. [16] presented a comprehensive autonomous drone system for search and detection operations, specifically covering search and rescue (SAR) and wildfire detection scenarios. Their approach integrates two key components: a path-planning algorithm based on Bayesian inference and a target detection system using a residual neural network (ResNet). The path planning algorithm optimizes the drone's route by utilizing probabilistic knowledge of target locations, while the ResNet-based detection system achieves more than 91% accuracy in identifying targets from aerial imagery. The system employs a sliding window technique to manage computational complexity and incorporates probability updating through Bayesian inference for improved search efficiency. Their system demonstrated superior performance through simulation and real-world experiments compared to benchmark algorithms, achieving significantly reduced search times and energy consumption. Their approach proved five times faster than traditional zigzag search patterns for a Gaussian mixture distribution scenario. The authors evaluated their system by using a DJI Mavic 2 Zoom drone equipped with a 4K camera, testing it in various conditions involving fire detection and person detection in snowy environments. While proving effective in these different scenarios, the authors note potential improvements through ensemble ML models and extension to multi-UAV operations.

2.3.2 SAR Operations in disaster environment with bio radars

Mittal et al. [17] presented a vision-based autonomous UAV navigation and landing system designed explicitly for Urban Search and Rescue (USAR) operations in post-disaster environments. The system enables UAVs with bio radars to autonomously navigate through and land on collapsed buildings to locate survivors. Their key contributions include (1) A novel landing site detection algorithm that evaluates multiple hazard factors, including terrain flatness, steepness, depth accuracy, and energy consumption through weighted cost maps; (2) An efficient exploration strategy using a next-best-view planner combined with TSDF-based 3D reconstruction for mapping

disaster environments; and (3) A minimum-jerk trajectory generator for safe landing maneuvers. The system ran in real time on an embedded computer. It was extensively validated both in simulation using their introduced synthetic dataset of 1.2 million images and in real-world experiments at a rescue training facility. Results demonstrated reliable detection of safe landing sites on rubble piles and debris, outperforming existing approaches that tend to Favor flat ground surfaces. The authors also released the first large-scale dataset of collapsed buildings with ground truth depth, surface normal, and semantic labels to facilitate further research.

2.3.3 Autonomous Motion Planning System for Outdoor Environments with Uncertainty in Object Detection Using Drones

Sandino et al. [18] presented a modular system for autonomous motion planning and object finding using small UAVs, focusing on search and rescue (SAR) missions. The system employs a Partially Observable Markov Decision Process (POMDP) to enhance UAV autonomy for obstacle avoidance, environment exploration, and object inspection under uncertainty conditions. The architecture integrates Software-in-the-Loop (SIL) for PX4 autopilot emulation and Hardware-in-the-Loop (HIL) for onboard inference using a MobileNet SSD detector and motion policy computation via TAPIR and ABT solver. The system was evaluated in simulation and preliminary real flights using three flight modes: mission (baseline), offboard (POMDP-based), and hybrid (combined approach). Testing involved locating a victim in two scenarios with varying occlusion levels. Results showed offboard and hybrid modes achieved superior detection accuracy (100% and 92.5% respectively) compared to the baseline, with minimal collision rates (1.25% overall) during navigation. The research notably demonstrated improved robustness against false positives and the ability to handle object detection uncertainty in outdoor environments using resource-constrained hardware, though flight times increased due to the comprehensive inspection approach. The study recommends future work including detailed real-flight analysis, evaluation with additional sensors like thermal cameras, and investigation of more complex scenarios with multiple or dynamic targets.

Chapter 3 Methodology



Figure 2 Methodology Overview

This research develops a systematic methodological approach to develop and evaluate drone path planning strategies within a grid-based simulation. The methodology encompasses environmental design, algorithm implementation, and performance evaluation frameworks to assess navigation performance under varying conditions. A structured experimental design enables direct comparison between classical and machine learning approaches, providing validated insights into their respective strengths and limitations for autonomous drone navigation applications. The core methodological framework integrates three navigation simulations: deterministic A* pathfinding, adaptive MR-Q-learning reinforcement learning, and a path optimization approach combining both methodologies. Each algorithm is implemented within a unified simulation environment featuring configurable obstacle distributions and dynamic movement patterns. This experimental setup allows consistent testing conditions while ensuring systematic variation of environmental complexity to evaluate algorithm robustness and adaptability.

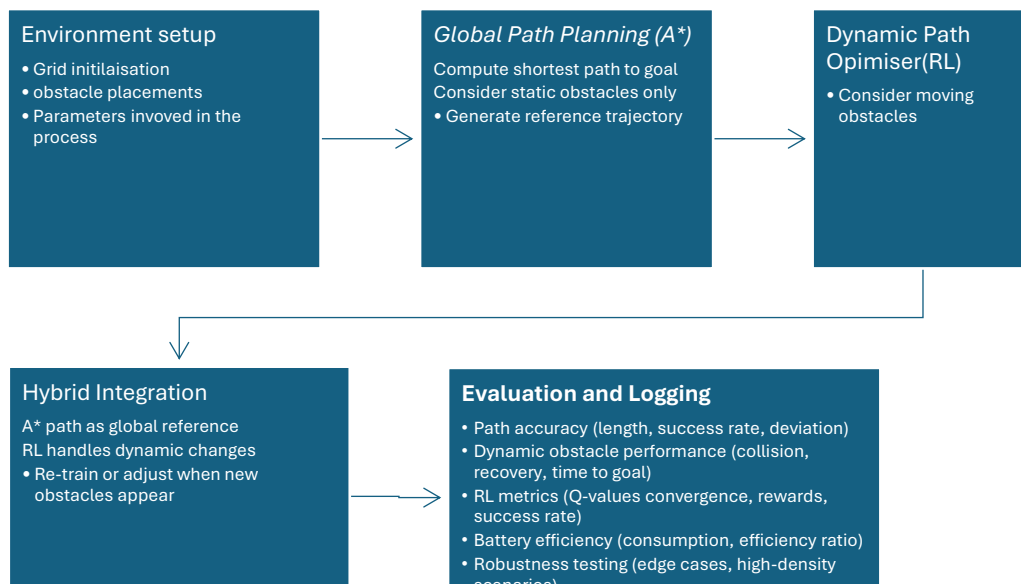


Figure 3 Research Components Diagram

3.1 Overall Workflow

The research establishes a complete workflow which unites the A* algorithm with improved MR-QLearning to create an integrated path planning system for drone autonomous flight. The workflow follows a step-by-step sequence which starts with environmental creation and ends with performance evaluation to develop and test the proposed hybrid method. The process begins with environmental setup which creates a 20×20 grid world with obstacles arranged according to defined density parameters. This setup is critical for consistent algorithm testing, evaluation, and validation. Following this, the system presents three operational modes through parallel simulation frameworks, which allow the user for direct visual comparison of the running algorithms. The system architecture implements three distinct operational modes through parallel simulation frameworks. The system architecture operates through three separate operational modes which use parallel simulation frameworks to deliver distinct algorithm performance assessments under different conditions. The first window runs the static global path planner using A* algorithm to measure its performance for optimal path generation and processing speed in areas with stationary obstacles. The second window demonstrates real-time adaptation through enhanced MR-QLearning in environments with static and dynamic obstacles to show learning-based navigation capabilities. The third window demonstrates the hybrid approach which uses A* global planning and MR-QLearning local adaptation to show how classical and learning-based methods work together for navigating complex environments. This three-model setup provides a comprehensive platform for analyzing the strengths and weaknesses of each algorithm whether it is the hybrid path planning approach or the Reinforcement Learning baseline or A* algorithm baseline. The workflow contains multiple optimization systems which work together to achieve mission success when operating in different environmental conditions. The system tracks battery efficiency through proportional energy consumption which causes total available capacity to decrease gradually during path execution. The system optimizes computational resources through data structure optimization and algorithm-dependent performance enhancements which include A* priority queues and Q-table storage using nested dictionaries. The assessment framework unites all operational algorithms through a single evaluation system which monitors their success rates and path length efficiency and battery usage.

3.2 Static Global Path Planner Based on A* Algorithm

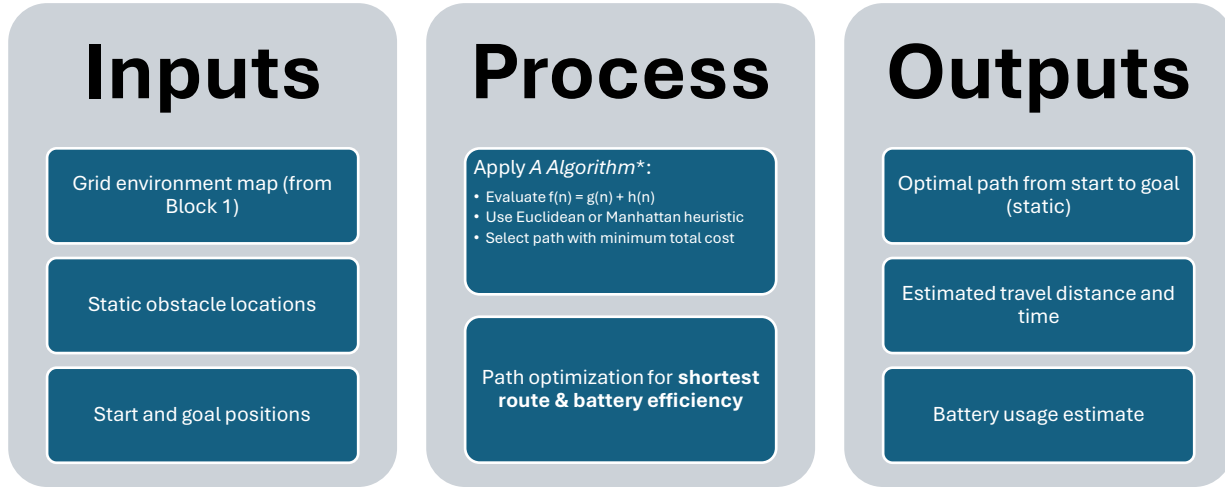


Figure 4 A* Algorithm

The static global path planner uses A* algorithm with search and rescue grid-based navigation optimizations to perform its implementation. The A* algorithm which operates on the grid environment to discover the shortest path between start and goal points while obeying static obstacle restrictions. The search process maintains computational efficiency through multiple essential data structures which support the implementation.

3.2.1 Algorithm Architecture and Data Structures

The grid representation uses a 20×20 2D list with binary values to represent free cells as 0 and obstacles as 1. The algorithm uses an open set priority queue based on Python's `heapq` module to store nodes for evaluation through their calculated f-score values. The `came_from` dictionary enables efficient path reconstruction by storing the predecessor of each node in the discovered path. The algorithm uses `g_score` and `f_score` dictionaries to store actual start-to-node costs and estimated total costs to the goal which enables efficient cost evaluations and optimal path determination.

3.2.2 Heuristic Function and Cost Calculation

The algorithm uses Manhattan distance as its heuristic function because this metric works best for grid environments where movements are limited to four cardinal directions. The heuristic function calculates the distance between two points using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ which

produces an underestimation of the actual path length to the goal. The algorithm uses a uniform step cost of 1 for each movement action between adjacent cells to maintain consistency with the discrete nature of grid-based navigation. The total cost calculation follows the basic A* formula $f(n) = g(n) + h(n)$ which combines the actual path length from the start to node n ($g(n)$) with the estimated path length from node n to the goal ($h(n)$). The unweighted heuristic function ensures algorithmic optimality because it provides the shortest path available in the grid environment. The simple cost system delivers dependable results while upholding the essential theoretical requirements for critical search and rescue operations.

3.2.3 Node Expansion and Neighbor Evaluation

The algorithm performs systematic search of space through repeated node expansion based on best-first search principles. The algorithm selects the node with the lowest f -score from the priority queue through heap operations to start the expansion process. The search ends immediately when the current node position matches the goal coordinates because this triggers the path reconstruction process. The algorithm checks neighboring cells in all four directions (up, down, left and right) from the current node to validate their status. The system checks each potential neighbor to confirm it stays inside the grid area ($0 \leq x < 20$, $0 \leq y < 20$) and functions as a walkable space (grid value = 0). The system performs cost assessments for valid neighbors by updating their g -scores and f -scores when it finds better paths which leads to continuous search optimization. The algorithm explores all accessible areas through its methodical expansion which directs the search toward the most advantageous areas.

3.2.4 Path Reconstruction and Validation

The algorithm performs backtracking to find the best solution path after reaching its target through the a dictionary. The reconstruction process starts at the goal node and then follows predecessor references to the start position until it reaches the beginning point to create the forward path from start to goal. The search phase requires minimal memory resources because this method enables efficient path retrieval after the search process finishes. The path validation system performs various checks to verify both safe and correct drone navigation. The obstacle avoidance confirmation step verifies that the generated path stays within free cells which have a grid value of 0 to prevent any contact with static obstacles. The system checks all path coordinates to verify they stay inside the 0 to 19 range of both axes. The system checks for continuous cell-to-cell movement

to prevent any teleportation or discontinuous jumps that would break the navigation sequence. The system performs complete verification to confirm that the generated path follows all rules and can be executed by the drone platform.

3.2.5 integration with Simulation Framework

The A* implementation works perfectly with the python based Tkinter library visualization system to show complete operational feedback and real-time observation of the algorithm. The visualization system shows animated path construction which demonstrates the algorithm's decision-making sequence while users can watch the search frontier grow and shrink during execution. The algorithm's navigation choices become easier to understand because the system uses different colors to show both navigable and blocked areas in the simulated environment. The system enables users to perform interactive testing through automatic environment changes which allow them to test different obstacle arrangements quickly. The visualization system uses different colors to show the start position (green) and goal position (red) and obstacles (black) and free space (white) and discovered path (blue) to create an easy-to-understand algorithm operation display. The system displays actual execution metrics in real time after the algorithm finishes its run by showing path distance and processing duration and success indicator. The algorithmic computation and visual representation work together perfectly to help users debug their code and present their results.

3.2.6 Computational Complexity and Performance

The algorithm produces stable computational patterns which make it suitable for real-time search and rescue operations. The algorithm explores all possible paths through its branching structure during worst-case scenarios but heuristic guidance helps to decrease the actual search area. The memory usage grows directly with grid dimensions because it needs to store cost information and path reconstruction information. The system provides guaranteed optimal solutions through shortest path discovery when solutions exist because this capability is vital for fast drone operations in search and rescue scenarios. The 20×20 grid environment shows sub-second processing times for all obstacle arrangements. The algorithm provides complete and optimal solutions which makes it suitable for building hybrid navigation systems.

3.3 Adaptive Navigator Via MR-Qlearning (Modified Reward)

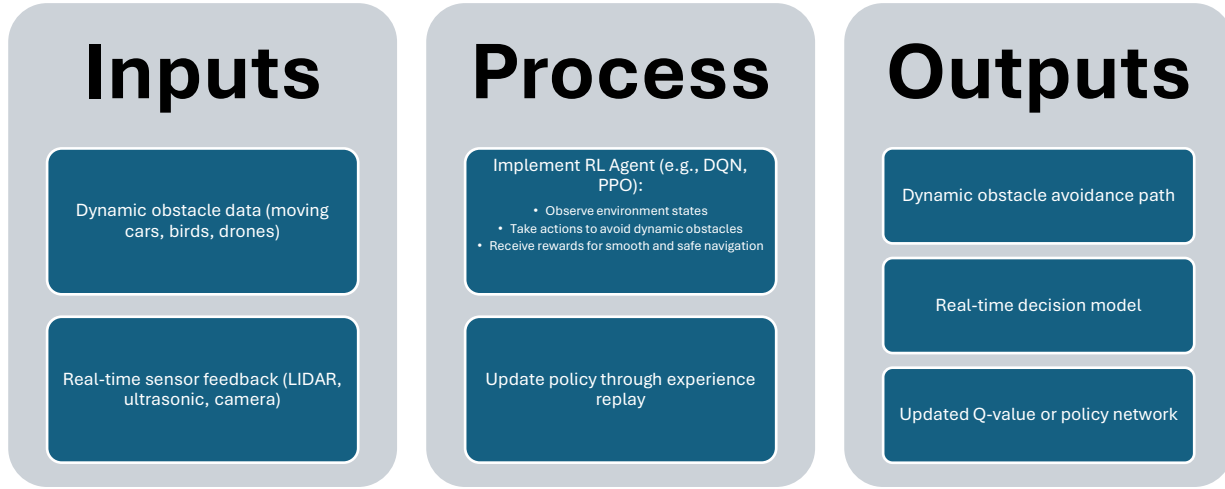


Figure 5 Reinforcement Learning Algorithm

The adaptive navigator module uses an improved MR-QLearning algorithm which adds reward shaping and heuristic-based exploration to let the drone discover adaptive navigation strategies by interacting with changing environments. The system uses reward shaping and heuristic-based exploration to enhance Q-learning for real-time decision-making and obstacle avoidance in unpredictable SAR scenarios. The MR-QLearning architecture operates through multiple essential elements which establish its complex learning system including a organized state representation and specified action options and Q-table data structure and sophisticated exploration mechanism.

3.3.1 Q-learning Architecture and Core Components

Q-learning architecture functions through some core components that define the learning framework. The state space includes all possible grid coordinates $S = \{(x, y) \mid 0 \leq x, y < \text{GRID_SIZE}\}$ which represent all locations the drone can access while navigating through the environment. The action space $A = \{\text{up, down, left, right}\}$ defines the four basic directions which the drone can move in the grid environment. The Q-table uses a nested dictionary structure to store state-action values under the format $\{\text{state: \{action: Q-value\}}\}$ which enables fast access during training and execution phases. The policy uses an epsilon-greedy approach with heuristic bias to explore new paths while exploiting known effective routes which results in a reliable learning system for changing environments.

3.3.2 State Representation and Action Selection

The system employs discrete state encoding with optimized decision systems which generate operational navigation decisions. The action selection method uses an improved epsilon-greedy policy which adds heuristic guidance during exploration to achieve better learning performance than random exploration. The algorithm selects actions based on probability during exploration by choosing moves that decrease Manhattan distance to the goal at rates that match the improvement in heuristic value. The method speeds up learning process by focusing exploration on beneficial areas of the state space while keeping enough randomness to find new solutions. The algorithm performs action selection from only accessible cells because it removes blocked cells from decision-making processes to ensure policy-level safe navigation. The system chooses random actions from unvisited states through biasing when it lacks path information to progress through unfamiliar environmental configurations while staying focused on reaching the goal. The advanced action selection system in this implementation surpasses basic Q-learning by delivering both high performance and stability in challenging navigation environments.

3.3.3 Reward Function Design

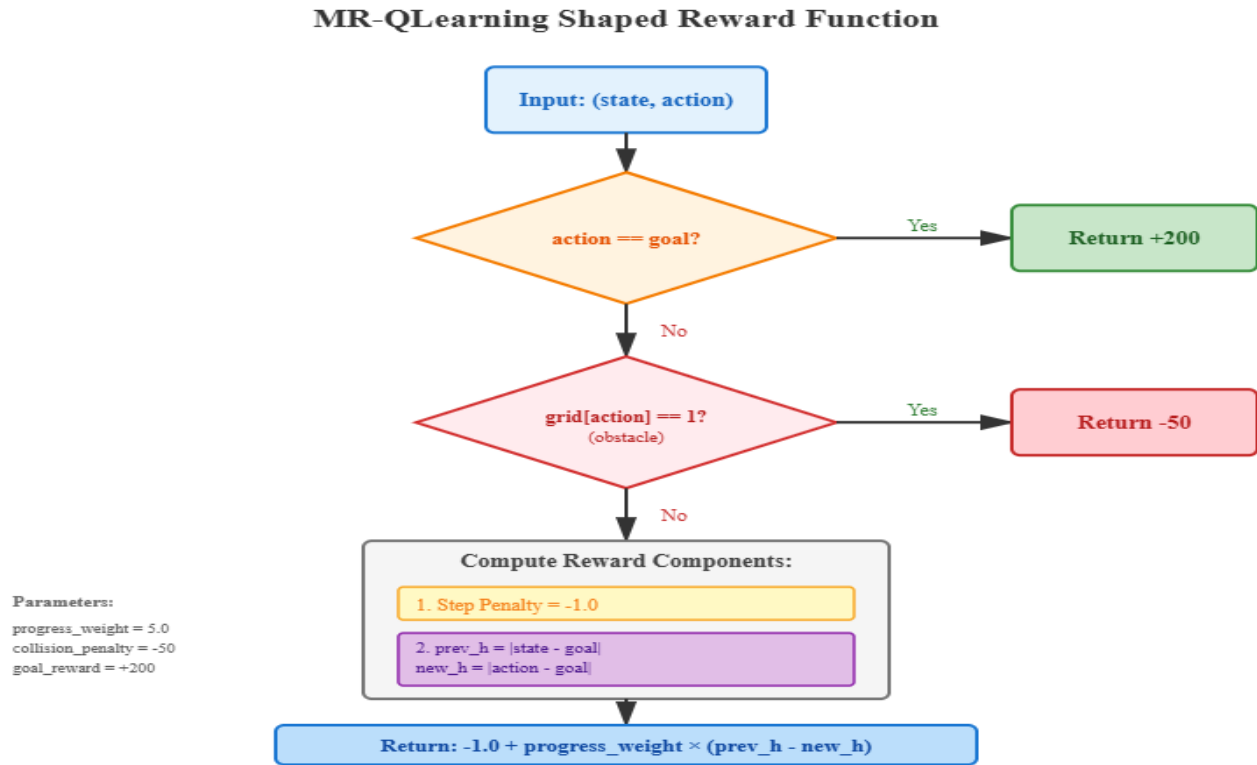


Figure 6 Flowchart Diagram of The Reward Function

The reward system undergoes precise development to fulfill multiple goals while sustaining stable learning performance through training sessions. The shaped reward function provides +200 points when the drone reaches its target position to deliver strong positive feedback for mission success. The reward function uses a base step penalty of -1 together with a progress bonus that depends on the decrease in heuristic distance to the goal at a weight of 5.0. The shaped reward system promotes both efficient route selection and straight path advancement to the target through its natural learning direction. The reward system implements a -50 penalty for attempted obstacle collisions, but this scenario becomes unnecessary because the action filtering system produces only valid movement options. The reward system provides complete coverage of desired behaviors through its design which rewards battery efficiency by reducing steps and ensures safety by enforcing valid actions and directs the agent toward its goal through progress-based rewards. The designed reward structure outperforms basic reward systems because it delivers detailed feedback that enhances learning speed and produces better final policy performance.

3.3.4 Training Process with Early Stopping

The training system enhances policy performance through episodic learning with early stopping mechanisms which perform systematic updates to achieve better navigation abilities. The training process runs for 300-350 episodes to achieve policy stability but includes an automatic early stopping mechanism which stops training when success rates reach 90% across 30 consecutive episodes. The method achieves optimal learning performance by stopping training after the policy reaches an effective solution point. The system executes real-time Q-value updates through standard temporal differences learning update rules during operational execution. The training system implements effective episode termination rules which activate when the agent achieves its target or hits the maximum step threshold to prevent endless loops in difficult situations. The policy development shows typical learning patterns because the agent learns successful strategies by interacting with the environment which leads to effective navigation. The Policy development follows a controlled path from wide exploration to specific exploitation of acquired knowledge because of epsilon decay and heuristic guidance which produces efficient and dependable navigation behavior.

3.3.5 Path Extraction and Execution Mechanisms

The trained policy becomes ready for Practical path planning tasks after training completion through which path extraction uses a greedy approach to select actions with the highest Q-value. The system performs policy execution through state-by-state action selection which chooses the best path based on the learned value function. The system uses real-time obstacle detection to validate all executed actions based on present environmental conditions while maintaining operational safety in dynamic environments. The system contains a secure fallback mechanism which activate A* algorithm when the learning system encounters untrained situations or when navigation fails to reach the target. The safety feature maintains mission reliability through its ability to handle unexpected learning-based approach challenges. The system uses step limiting to stop infinite loops in complex environments while establishing operational boundaries for difficult situations. The system unites learned policies with classical fallback methods to achieve machine learning adaptability and proven algorithm reliability.

3.3.6 Learning Parameter Optimization

The MR-QLearning algorithm needs specific optimization of its search and rescue mission parameters to achieve maximum performance through its Q-value update rule which follows the standard temporal difference learning formula $Q(s, a) = (1 - \alpha) * Q(s, a) + \alpha * [\text{reward} + \gamma * \max_{a'} Q(s', a')]$. The learning rate α sets to 0.7 which positions itself above standard learning rate values to determine the amount of new information that should replace existing knowledge. The system adapts quickly to changing conditions through its aggressive learning rate which combines with existing knowledge to maintain stability in dynamic environments. The discount factor γ at 0.9 shows that the system prefers future rewards over immediate rewards which supports successful completion of long missions that need strategic planning and multi-step planning. The drone starts with an epsilon value of 0.25 to select random actions for route discovery at 25% rate but uses 75% of its decisions to follow known paths. The exploration rate starts at 0.25 but decreases through time using a decay factor of 0.995 until it reaches 0.05 which establishes an optimal balance between exploration and exploitation. The system reaches convergence after 300-350 training episodes in the 20×20 grid environment but early stopping mechanisms can shorten this time when the system demonstrates consistent success.

3.3.7 Integration with Dynamic Environment

The reinforcement learning module directly interacts with environmental elements that change in real time to prove its ability to adapt to dynamic situations. The system updates its current situation awareness through real-time state updates which respond to every modification in the grid environment during execution to produce decisions based on present world conditions. The obstacle reactivity function of the system enables it to detect and steer clear of new environmental obstacles which makes it suitable for handling unexpected search and rescue situations. The algorithm shows policy generalization capabilities which allow it to use acquired behaviors for different obstacle configurations in real-world operations. The system achieves generalization through its organized state encoding and its guidance system which teaches essential navigation rules instead of teaching particular routes. The system proves suitable for real-world deployment because it operates within dynamic environments that experience unpredictable changes during missions.

3.4 Path Optimization by Combining A* and MR-QLearning Approach

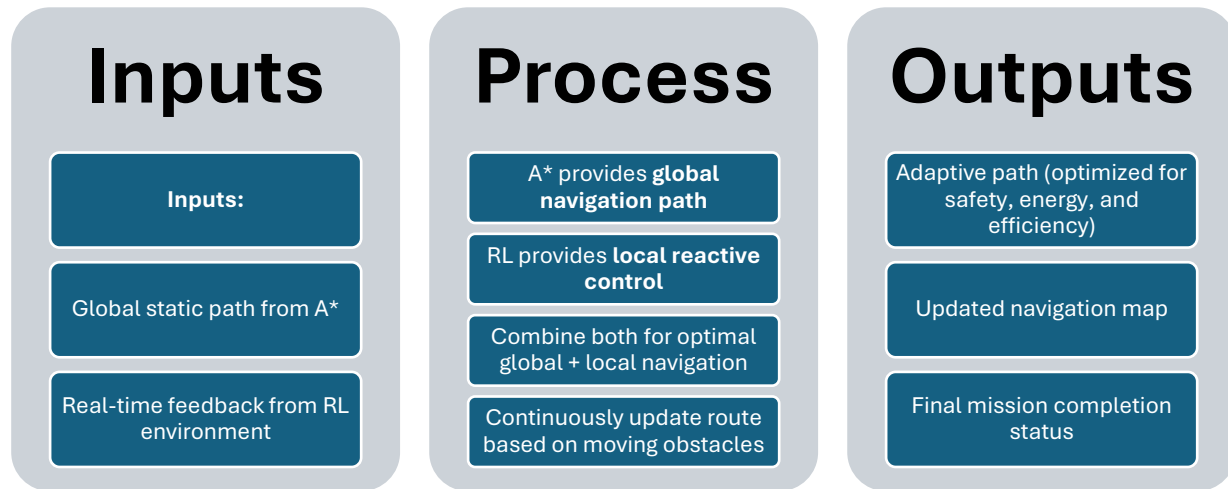


Figure 7 Combination Approach

The Combining of A* and MR-QLearning approach serves as the main contribution of this research because it unites traditional path planning techniques with machine learning algorithms to create an efficient navigation system for search and rescue operations in dynamic environments. The system uses a two-layer design which divides path planning responsibilities between A* for initial route discovery and MR-QLearning for local obstacle avoidance. The system achieves complete

navigation coverage through its separate components which operate at their peak performance levels.

3.4.1 System Architecture and Components

The Path optimization system uses modular design with established components and communication standards to achieve smooth operation through various environmental settings. The A* algorithm in the global path planner determines the best route by analyzing static obstacles during mission setup, using grid environment and start and goal positions to generate an optimal coordinate sequence based on static conditions. The local adaptive planner uses MR-QLearning functions as its technical adaptation layer to handle unexpected environmental changes, which activates automatically during mission execution when dynamic obstacles appear instead of starting at mission beginning.

3.4.2 Global Path planner implementation

The global path planner component runs the A* algorithm with grid-based navigation optimizations to generate the basic path which the hybrid system tries to execute under perfect circumstances. The component runs during the planning stage before starting the mission to study the fixed environment for finding the best path to the goal while skipping all recognized barriers. The A* algorithm uses Manhattan distance as its heuristic function because it works best for four-directional grid navigation to produce admissible heuristics that ensure optimal solutions. The global planner uses priority queues (heapq) for open set management and dictionaries for cost tracking to achieve efficient computation in extensive environments. The hybrid system uses this path as its reference trajectory to achieve optimal performance in static conditions. The system uses its global understanding to stay focused on the target even when it needs to make local adjustments which helps prevent suboptimal total path performance. This global coverage keeps the system goal-directed with the overall path optimality

3.4.3 Local Adaptive Planner

The MR-QLearning algorithm operates within the local adaptive planner to handle situations that make the precomputed global path unusable because of moving objects or changing environmental conditions. The system activates this component through automatic operation during mission execution when it detects obstacles that block the next waypoint from the initial planning phase.

This check is performed by verifying if the next step is clear before the drone attempts to go to the wanted cell. The local planner maintains a limited operational area which directs the system to find the following valid waypoint on the global path or to proceed directly to the goal when it becomes accessible. The local planner conducts focused training through 250 episodes because it needs to learn navigation strategies for the local area instead of the complete environment. This is implemented by initializing a new MRQLearning object with episodes=250 exactly at the point of obstruction, training it on current grid state from the drone's present location (state) to the goal. The hybrid system achieves real-time operation through its reduced training needs, which still enable adaptive behavior. The local planner uses MR-QLearning's shaped rewards together with heuristic-based exploration to find suitable local routes when encountering unexpected obstacles.

3.4.4 Dynamic obstacle avoidance Mechanism

The pure A* algorithm fails to perform obstacle avoidance when dealing with moving or unexpected obstacles in dynamic environments. The path optimization approach resolves this problem through real-time adaptation capabilities which reinforcement learning activates during specific situations. The system checks every step of its executed path to ensure the following position remains clear of obstacles by comparing the current environment state to the planned trajectory. This nonstop monitoring is a fundamental part of the `animate_hybrid()` approach main loop, which runs for each step in the global path. The system maintains its awareness of environmental changes throughout mission execution because of its continuous monitoring system. The system activates the local MR-QLearning agent through optimized parameters when it detects a moving obstacle that blocks the planned route. The local training system teaches the agent to move from its present location to either continue following the global path past the obstacle or to directly reach the goal when a more efficient route exists. The system uses this focused method to deal with moving obstacles because it prevents extensive global planning computations while keeping the mission progress intact. This design choice is evident in the code, which only replans the remaining segment of the path, which is better than recalculating from the start, keeping mission efficiency. The system achieves complete obstacle avoidance through its combination of global planning and local adaptation for both static and dynamic environments, which mirrors the scenarios that would happen in a real-world search and rescue mission.

3.4.5 Fallback to A* Mechanism

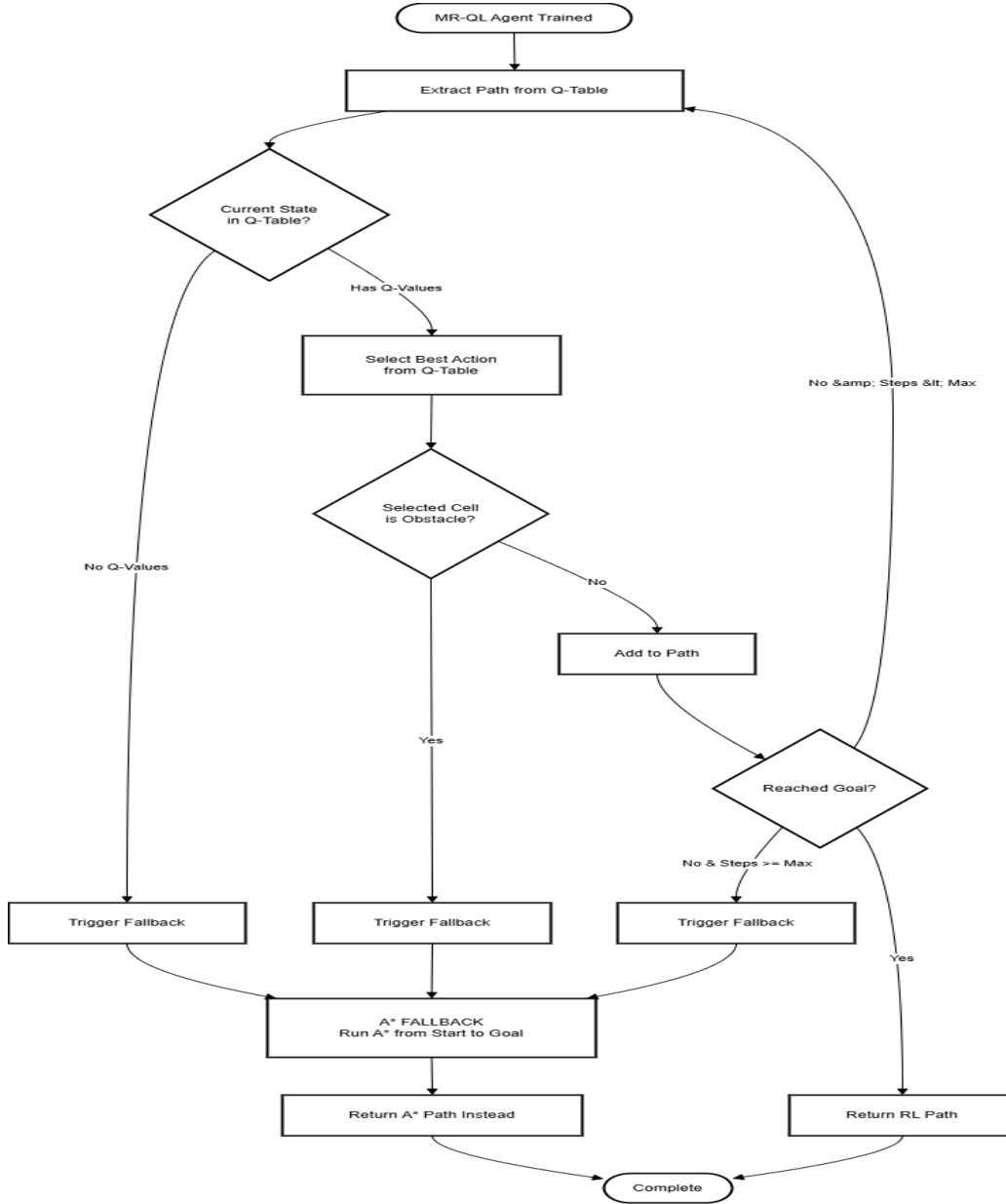


Figure 8 Flowchart Diagram of the A* Fallback Mechanism

The fallback mechanism operates as a vital safety function which guarantees system dependability through A* algorithm execution when MR-QLearning agent faces navigation breakdowns or when local learning fails to produce results. The system maintains operational capability through challenging situations because learning-based methods face limitations when training data is insufficient or when obstacles become too complex. The system activates the fallback mechanism when the local planner fails to discover a path within its processing capacity or when goal

progression stops moving forward past a specific threshold. This occurs when the local MR-QLearning path planning returns an empty or invalid path, triggering the instantiation of a new AStar object from the drone's current position. The system performs a complete global replanning operation starting from the present location while using the updated environment information to create a new navigation plan. This will make a recalculates the entire path based on the current obstacle configuration. The system maintains its expected reliability in critical search and rescue operations because this complete fallback mechanism prevents permanent system failure from depending too heavily on learning-based components. The hybrid system achieves its main benefit through its ability to switch between learning-based adaptation and classical planning which delivers sophisticated functionality with dependable performance.

3.4.6 Integration System and Coordination

The integration system controls planning strategy transitions through real-time environment data which serves as the fundamental coordination system that enables A* and MR-QLearning to function together. This coordination is centralized in the code, specifically within the path optimization hybrid method, which serves as the main control loop for the entire hybrid system. The system tracks environment states and algorithm results and mission status to decide when to use each planning method. It maintains critical state variables including the current state, the path index, and continuously monitors the grid for changes and checks at each iteration. The system maintains state information about current location and path conditions and environmental updates which enables smart algorithm selection. The coordination system uses predefined rules to decide when to activate local adaptation or global replanning while optimizing between processing speed and solution quality. The system tracks operational performance metrics which help determine algorithm selection through success rates and path efficiency and computational expense measurements. The hybrid system achieves better results than single method approaches because of its intelligent coordination system which preserves operational reliability and efficiency for practical search and rescue operations.

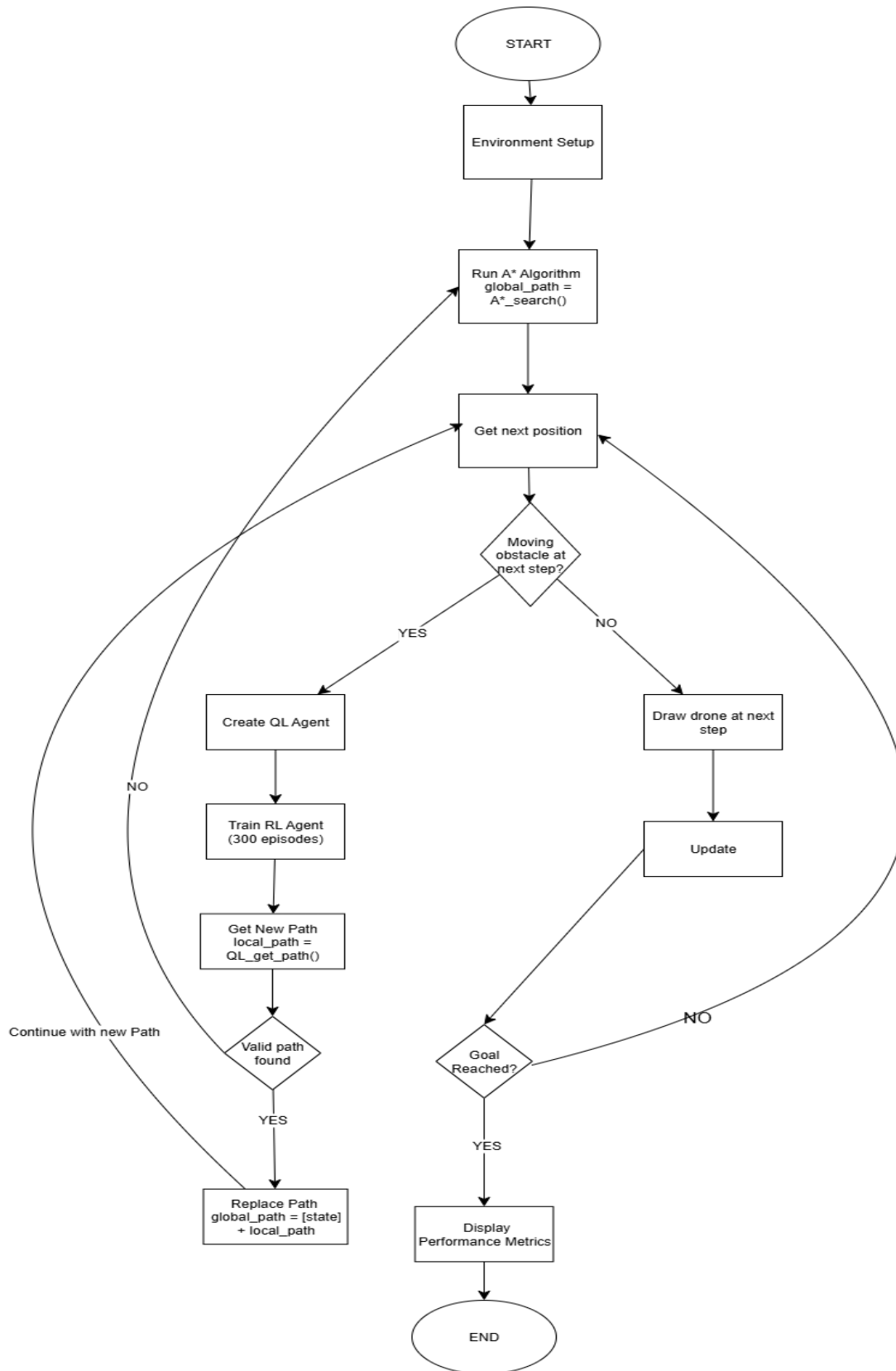


Figure 9 Path Optimization Approach Flowchart

Chapter 4 System and Environment Implementation

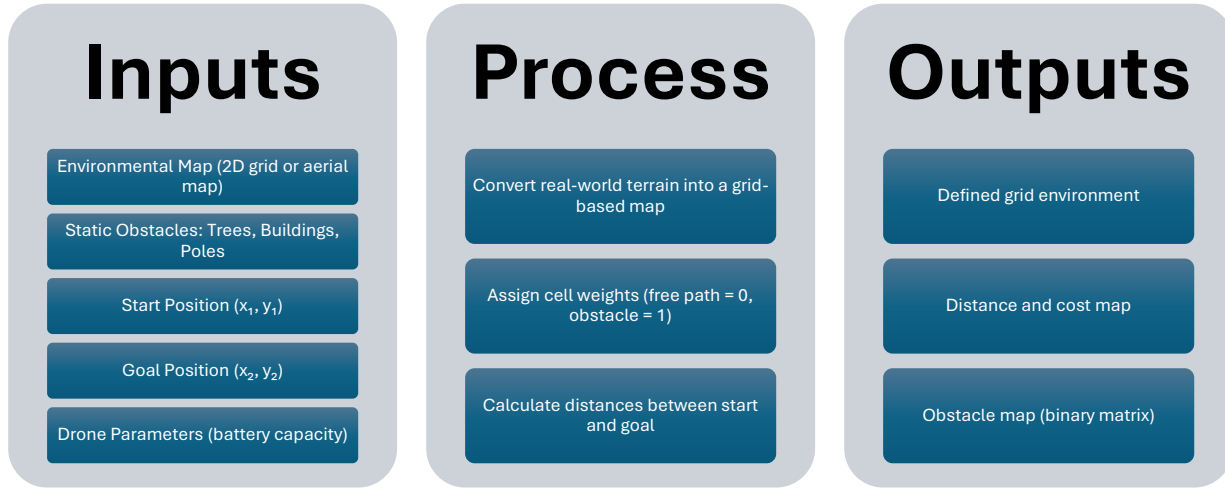


Figure 10 Environment Setup

The following chapter explains the technical details of two separate simulation platforms which were created to test and show the proposed path optimization system. The first environment operates as a development platform which allows users to watch the hybrid system work alongside baseline methods through independent viewing windows. The testing environment operates as a controlled space which establishes standardized experimental settings to compare all algorithms under equal environmental conditions. The system development and demonstration and systematic evaluation utilize Python with graphical interface for visualization which provides dual functionality for system development and demonstration and evaluation.

4.1 Development and Demonstration Environment

The development environment contains four main components which work together as a modular system. The environment manager controls both grid visualization and obstacle placement functions. The algorithm implementations contain basic pathfinding and reinforcement learning functionality. The visualization system enables users to view graphical content and interact with the system through its interface. The simulation controller maintains control over component execution throughout the system. The system design allows developers to test individual components separately while maintaining the ability to merge them into the complete system. The system allows users to view different pathfinding methods side by side through separate windows. The environment uses a 20×20 grid structure which exists as a two-dimensional data structure.

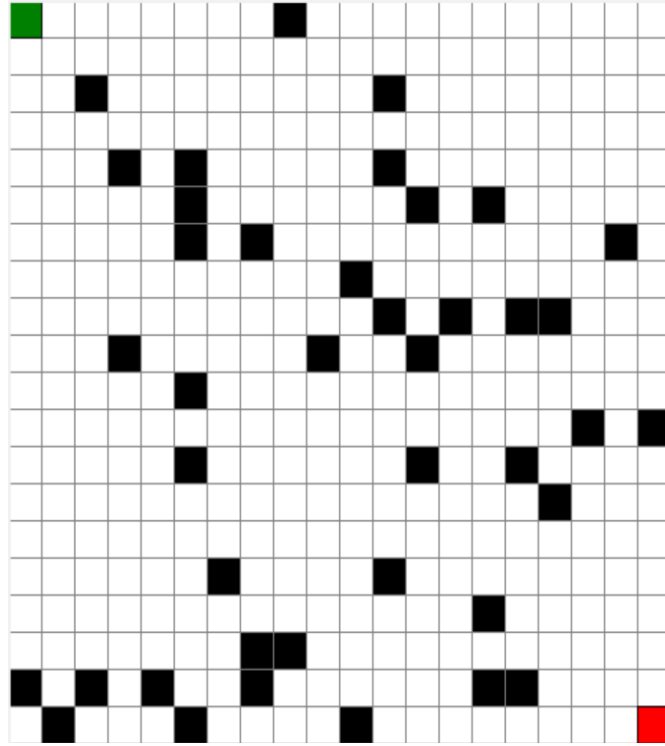


Figure 11 Development and Demonstration Grid Environment

The cells in this structure use binary values to show whether they are available for navigation or if they contain obstacles. The obstacle generation system creates obstacles that occupy 25% of the grid space while performing validation tests to confirm path accessibility. The game starts at position (0,0) and ends at position (19,19). The generation algorithm checks for multiple paths between the starting point and ending point before starting the path creation process. The system implements a backup system which removes a confirmed path to achieve environment solvability while preserving authentic obstacle placement. The A* implementation achieves optimal path finding through its priority queue-based node exploration system and its use of Manhattan distance heuristics. The reinforcement learning system uses Q-Learning with epsilon-greedy exploration and reward shaping to learn from the environment. The system uses a learning rate of 0.7 and a discount factor of 0.9 with an initial exploration rate of 0.25 that decreases exponentially. The training process consists of 300-350 episodes which stops when success rates reach 90% across 30 consecutive episodes. The two components use the same grid representation to maintain environment consistency during their operations.

The visualization system generates graphical windows which show grid cells through color coding: white indicates free space while black shows obstacles and green marks the start position and red

indicate the goal position and the algorithm uses different colors to show path execution (blue for A* and green for Q-Learning and purple for Hybrid). The interface features speed control which lets users select animation step times between 20 and 1000 milliseconds to study decision-making in detail. The interface contains buttons which let users start simulations and restore the environment to its initial state. The system shows immediate execution status through its real-time feedback displays. The system allows users to run different algorithms side by side through its independent window feature. The environment generates dynamic obstacles which amount to 10% of total obstacles during generation. The obstacles between animation frames choose their movement direction randomly from cardinal directions while staying inside the grid space and keeping away from both start and goal points. The system checks obstacle positions to verify they stay within available cells while avoiding contact with stationary objects. The system generates authentic environmental unpredictability which does not impact mission success to evaluate algorithm performance when facing unexpected changes during execution.

4.2 Testing and Evaluation Environment

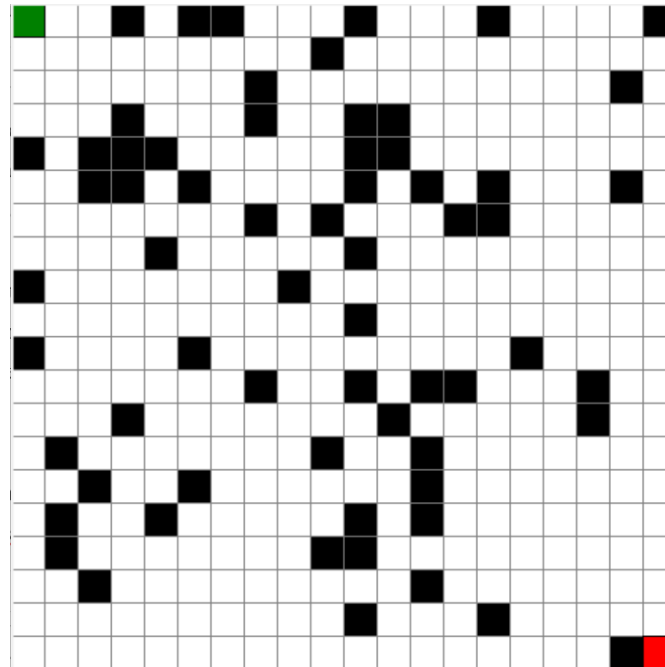


Figure 12 Testing and Evaluation Environment

The testing environment operates through centralized environment creation and deployment which differs from the development platform that uses individual simulator configurations. The central

controller produces one master grid configuration which distributes to all three algorithm simulators for execution. The system design removes environmental differences as a source of interference because it generates identical configurations for all algorithm simulators which allows performance differences to stem from algorithmic properties. The system includes automated output directory management to achieve organized result storage.

4.2.1 Environment State Management

The central controller produces one master environment with 15% obstacle density which exceeds the 10% level used during development to create complex testing scenarios and it establishes a common database for moving obstacles. The system distributes complete copies of grid configuration and obstacle positions to each simulator instance. The operational grid of each simulator remains subject to change while it keeps an unalterable reference to the master configuration. The reset functions of the system enable environment restoration through master configuration copying and obstacle position resetting and metric clearing which allows statistical analysis through repeated trials of the same configurations.

4.2.2 Performance Metric Collection

The testing environment tracks all real-time metrics throughout its operation. The simulator tracks two types of collisions through its counters which monitor both dynamic encounters with moved obstacles and static encounters with original obstacles. The system tracks time duration from simulation start until the goal becomes reachable. The battery starts at 100 units and decreases by 0.4 units for each movement step. The step counter increases whenever the system performs a movement action. The display system presents these values in a formatted way for continuous display while updating after each animation frame to show real-time performance data.

4.2.3 Collision Detection System

The system performs pre-movement validation to operate its collision detection function. The system performs a target cell occupancy check before it updates positions. The system increments the collision counter when obstacles block target cells. The system uses a reference set based on the master grid configuration to identify collision types at high speed. The system uses two different collision counters for obstacles in the reference set because static collisions occur when obstacles exist in this set while dynamic collisions occur when other obstacles are present. The system

separates algorithm performance evaluation between different obstacle types to show how well the system adapts versus its planning precision.

4.2.4 Real-Time Display System

The visualization system updates its display in real-time as the program executes. The display system generates formatted text that shows current metric values after each movement step. The display system uses proper decimal places for its numerical values which include three decimal points for timing measurements and one decimal point for battery percentage and whole numbers for step counts and collision events. The system displays formatted data through specific display sections which update instantly to show real-time performance data and algorithmic behavior patterns.

4.2.5 Multi-Window Coordination

The testing environment displays three simulator windows at specific screen positions which allow users to observe all three screens at once. The system runs A* followed by Q-Learning and then Hybrid approach in sequential order. The main control window displays status updates between each execution cycle. The system runs trials in sequence because it finishes each trial before starting the next algorithm to prevent interference while keeping the environment stable. The system regenerates the master environment when it generates a new map and uses reset mechanisms to distribute changes to all simulators for testing different system configurations.

4.2.6 Hybrid System Integration

The hybrid implementation allows the system to follow its initial global paths while it continuously checks the surrounding environment. The system checks for available positions at each step of its operation. The system starts local reinforcement learning when it detects blocked waypoints to find new routes starting from the current position. The system uses new path segments that result from successful local adaptation to connect with the remaining global plans. The system performs full global planning from the current position when local adaptation fails. The system uses a dual system to achieve global optimization when possible while it adapts to environmental changes through local adjustments.

4.2.7 Key Implementation Differences

The development environment supports fast prototyping and visual proof through independent simulators which generate their own environments. The testing environment focuses on precise measurement and controlled experiments through its shared environment setup and complete metric recording system and defined reset functions. The testing environment contains fewer obstacles than the development environment because it needs to show clear results while maintaining complex evaluation conditions. The testing environment uses fixed battery costs of 0.4 units per step, but development uses proportional battery consumption to support thorough analysis requirements.

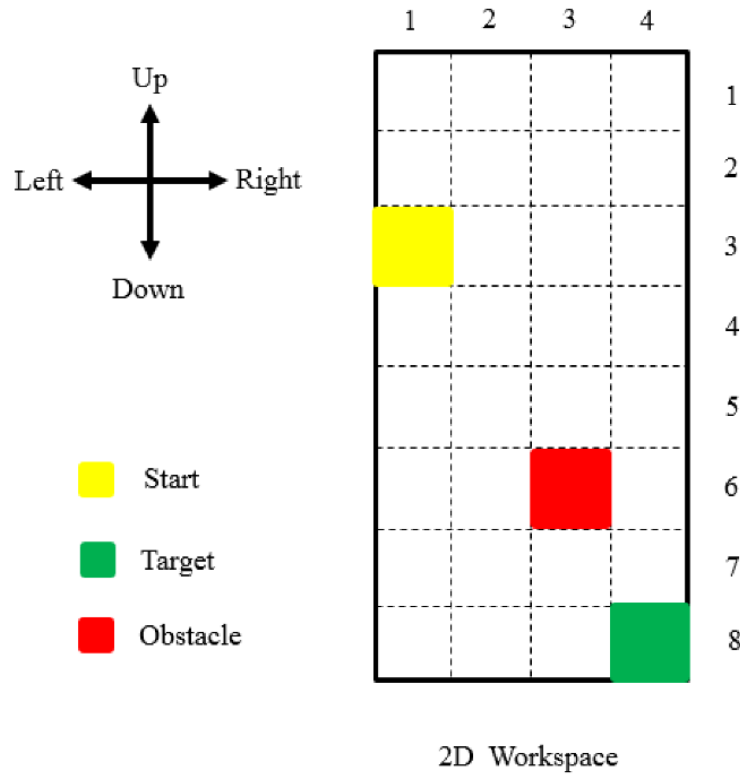


Figure 13 workspace including start, target, obstacle from the top view, as well as the actions [25]

The systems operate with Python 3.x and include all standard library elements. The graphical interfaces use tkinter library to create canvas widgets for grid display and label widgets for information display and button widgets for user controls and scale widgets for speed adjustment and frame widgets for layout organization. The system generates random numbers to create stochastic elements which control action selection and generate obstacles and create dynamic movement patterns. The timing system enables users to measure execution time and control

animation delay periods. The system uses priority queues to optimize its pathfinding operations. The testing environment uses file system operations to handle output directory management. The system uses a basic set of dependencies which allows it to run on multiple platforms with easy deployment on different computing systems.

Chapter 5 Experiment and Results

The experimental evaluation of the proposed path optimization method occurs in this chapter through controlled simulation environment testing against baseline algorithms. The evaluation framework consists of multiple performance metrics which measure algorithm effectiveness through navigation efficiency and energy usage and collision prevention and time-based performance. The experimental design uses the testing environment from Section 4.2 to achieve fair results through centralized environment creation and uniform metric collection for all tested algorithms. The chapter consists of two main sections. The testing environment describes the five-performance metrics in Section 5.1 which detail their technical implementation and definition. Section 5.2 presents experimental findings from testing the hybrid A*-MR-Q-Learning method against A* and Q-Learning algorithms through systematic evaluation in different environmental settings.

5.1 Performance Metrics

Table 2 Testing Parameters

Parameter	Description
Environment	2D grid with static and moving obstacles
Algorithm	Path Optimization by Combining A* and MR-QLearning Approach Reinforcement Learning Q-Learning Baseline A* Algorithm Baseline
Maps Tested	10 unique map configurations
Agent Metrics Recorded	Steps taken, static Collisions, Dynamic collisions, Time to Goal, Battery remaining
Evaluation Type	One trial per map with real-time visualization
System Used	Python 3.10+

The testing environment uses five performance metrics which it automatically records during simulation execution. These evaluation metrics offer comprehensive assessment of navigation effectiveness across multiple operational dimensions. Each metric has specific aspects of the performance to evaluate allowing fair comparison through the multiple environmental scenarios

5.1.1 Step Taken

The steps metric measures the complete number of individual cell movements that occur between the starting point and ending point. The step counter increases by one unit whenever the system moves from one grid cell to the adjacent cell. The metric directly measures path length while serving as the main factor to evaluate navigation efficiency. The number of steps in a path directly affects both the length of the route and the total time needed for completion. The testing environment uses the direct path from (0,0) to (19,19) which requires 39 steps to establish an evaluation standard for algorithm performance.

5.1.2 Battery Remaining

The battery remaining metric shows how much energy the system uses during operations by displaying its current capacity level as a percentage of the starting value. The simulation starts with a fully charged battery at 100% capacity which decreases by 0.4% after each movement operation. The proportional depletion model tracks energy usage at a constant rate for all algorithms. The navigation system achieves better energy efficiency when the battery percentage remains high. The battery remaining percentage shows the available energy level through the calculation $100\% - (\text{steps} \times 0.4\%)$. The measurement plays an essential role in assessing operational sustainability because real-world missions face energy restrictions that restrict flight time.

5.1.3 Dynamic and Static Collisions

The system uses collision metrics to measure how obstacles interact with the navigation system based on their specific types. The system tracks dynamic collisions which occur when obstacles shift their position after the initial path planning process because this indicates difficulties with environmental changes. The system tracks static collisions which occur when obstacles maintain their original positions from the beginning of the mission to detect planning or execution problems. The system determines collision types through a comparison process between current obstacle positions and the environment state from the mission beginning. The system achieves perfect performance when no collisions occur, but high collision numbers indicate insufficient planning or adaptation capabilities. The system tracks collisions independently to evaluate algorithm performance against various environmental conditions.

5.1.4 Time to Goal

The time to goal metric measures total time that the drone takes to reach the goal. The timer starts when the evaluation simulation begins and stops when the drone reach the goal position. Faster mission completion indicated by lower time values, critical for time-sensitive search and rescue operations. The metric shows the combined performance of path quality (shorter paths run faster) and computational performance (efficient algorithms reduce processing delays).

5.1.5 Mode Indicator

The operational mode indicator shows which algorithm produced each result set. The system operates in three distinct modes which include 'A*' for the baseline optimal pathfinding algorithm and 'RL' for the reinforcement learning baseline and 'Hybrid' for the combined path optimization approach. The identifier allows researchers to link performance characteristics directly to their corresponding algorithms during multi-algorithm evaluation. The mode indicator appears in all result displays and tables to prevent confusion when analyzing multiple algorithm results at once.

5.1.6 Metrics collection and Display

The testing environment runs all five metrics automatically throughout each trial execution. The system updates metrics in real-time during algorithm execution until it reaches mission completion to record the final values. The display system shows real-time current values of operational mode and steps taken and dynamic collision count and static collision count and elapsed time with three decimal places and battery remaining with one decimal place. The system provides continuous feedback which allows users to track performance in real-time during the entire execution process. The system resets all metrics to their starting values when beginning each new trial to achieve clean measurement results for algorithm assessments. The five metrics work together to deliver complete evaluation of algorithms. The two metrics of path length and energy help users determine how well their algorithms use resources. Collision metrics help users understand both safety performance and their system's ability to adapt. The time metrics help users determine how quickly their system operates. The operational mode system helps users identify the correct results. The combination of these performance metrics allows users to conduct detailed analysis of system performance while identifying key trade-offs between different objectives to select the best algorithm for their specific needs.

5.2 Experimental Results and Comparative Analysis

The following section shows experimental results from testing the hybrid A*-MR-Q-Learning path optimization method against two reference algorithms which include A* pathfinding and MR-Q-Learning reinforcement learning. The evaluation used the testing environment from Section 4.2 to achieve fair comparison through centralized environment generation which produced identical obstacle configurations for all algorithms. The testing process included multiple environmental scenarios with different obstacle densities and dynamic obstacle behaviors to evaluate algorithm performance in various operational conditions that match real-world search and rescue scenarios.

5.2.1 Performance Results by Algorithm

Table 3 Experimental Results Across 10 Environment Configurations

Environment	Algorithm	Steps	Dynamic Collisions	Static Collisions	Time (s)	Battery (%)	Obstacles (Static/Dynamic)
1	A*	39	0	0	6.353	84.4	58/8
	RL	41	0	0	6.800	83.6	
	Hybrid	39	0	0	6.375	84.4	
2	A*	39	1	0	6.338	84.4	56/6
	RL	39	1	0	6.286	84.4	
	Hybrid	39	0	0	6.252	84.4	
3	A*	39	1	0	6.273	84.4	59/4
	RL	501	0	0	83.020	0.0	
	Hybrid	39	0	0	6.348	84.4	
4	A*	39	0	0	6.272	84.4	69/8
	RL	41	0	0	6.668	83.6	
	Hybrid	42	0	0	6.922	83.2	
5	A*	39	1	0	6.289	84.4	67/3
	RL	41	0	0	6.667	83.6	
	Hybrid	39	0	0	6.375	84.4	
6	A*	39	0	0	6.270	84.4	47/2
	RL	41	0	0	6.593	83.6	
	Hybrid	39	0	0	6.339	84.4	
7	A*	39	1	0	6.291	84.4	60/7
	RL	43	0	0	6.965	82.8	
	Hybrid	39	0	0	6.328	84.4	
8	A*	39	0	0	6.284	84.4	49/3
	RL	43	0	0	7.067	82.8	
	Hybrid	39	0	0	6.324	84.4	
9	A*	39	2	0	6.268	84.4	57/7
	RL	43	0	0	7.024	82.8	
	Hybrid	39	1	0	6.370	84.4	
10	A*	39	2	0	6.358	84.4	53/5
	RL	41	0	0	6.737	83.6	
	Hybrid	39	0	0	6.378	84.4	

The above table presents the experimental results of the hybrid Path optimization approach and the baselines obtained from systematic testing of these algorithms. The evaluation employed the testing environment mentioned in 4.2. This environment ensured fair comparison and evaluation through centralized environment generation providing identical obstacle configuration to all algorithms. Testing encompassed multiple environmental scenarios with different obstacle density by randomizing the number of dynamic and static obstacles based on the requirement of the evaluation, representing the challenges of real-world search and rescue missions. The experimental data shows that the three algorithms demonstrate different operational behaviors. The RL algorithm failed catastrophically in Environment 3 by using 501 steps to drain all battery power (0.0% remaining) while getting stuck in a local minimum or encountering unknown situations. The A* algorithm and Hybrid approach both achieved optimal 39-step solutions through the difficult environment.

5.2.2 Comparative Analysis

Overall Performance Comparison: The Hybrid approach produced 39-step paths that were optimal in 90% of trials across 9 out of 10 environments. The Hybrid approach achieved 100% optimality in static planning just like A* but outperformed RL by achieving 22% optimality rate in 2 out of 9 successful trials except for Environment 3. The single deviation in Environment 4 reached 42 steps which represented a 7.7% increase from the optimal path in the scenario with 69 static and 8 dynamic obstacles.

A* Algorithm Performance: The A* Algorithm achieved optimal 39-step paths in all 10 environments showed perfect static planning optimality. The system detected five out of ten environments (Env 2, 3, 5, 7, 9, 10) experienced dynamic collisions which resulted in a maximum of two collisions in environments 9 and 10. The system required an average of 6.290 seconds to reach its destination while showing a standard deviation of 0.024 seconds. The system-maintained battery efficiency at 84.4% throughout all testing sessions.

RL Algorithm Performance: The RL system generated paths that spanned from 39 to 43 steps but excluded Environment 3 failure. The RL system achieved optimal 39-step paths in two out of nine successful trials which occurred in Env 2 and Env 10. The system required an average of 41.4 steps to reach its goal while showing a standard deviation of 1.4 steps. The system achieved complete obstacle avoidance during all successful runs because it demonstrated exceptional real-time

capabilities to handle moving obstacles. The system required 6.756 seconds to reach its goal with a standard deviation of 0.261 seconds. The system maintained battery efficiency between 82.8% and 84.4% with an average of 83.6%.

Hybrid System Performance: The Hybrid system produced optimal 39-step solutions in nine out of ten tested environments. The system produced a 42-step path in Environment 4 which represented a 7.7% increase above the optimal solution yet still achieved near-optimal performance. The system encountered one dynamic collision throughout its ten trials in Environment 9 while showing better obstacle avoidance than A*. The system required 6.361 seconds to reach its goal with a standard deviation of 0.027 seconds. The system operated at 84.4% battery efficiency in nine tests and 83.2% in Environment 4 which resulted in an average of 84.3%.

Collision Avoidance Analysis: The Hybrid path optimization system outperformed A* in dynamic obstacle avoidance because it experienced only one collision during its ten trials (10% collision rate) while A* experienced nine collisions in ten trials (90% collision rate when collisions occurred). The RL system prevented all collisions during its successful runs but its ability to avoid collisions became irrelevant because of its failure in Environment 3. The system successfully avoided all static collisions throughout all tests because the initial path planning correctly identified known obstacles. The Hybrid system's local adaptation mechanism proved better than A*'s pre-computed path following because it successfully avoided most moving obstacles. The Hybrid system encountered one collision in Environment 9 which proved to be the most difficult dynamic environment but A* experienced two collisions in this environment.

Timing and Battery Efficiency: The system achieved the fastest completion time through A* which took 6.290 seconds followed by Hybrid at 6.361 seconds (1.13% slower) and RL at 6.756 seconds (7.4% slower than A*). The Hybrid path optimization approach required an additional 71 milliseconds on average for its environment monitoring system and occasional path adjustments. The system's monitoring costs remain insignificant when compared to its improved collision prevention capabilities. The Hybrid system achieved better performance than A* in Environment 2 by completing the task in 6.252 seconds instead of 6.338 seconds which showed that effective adaptation can offset monitoring expenses. The RL system produced longer paths than optimal because it took an average of 2.4 extra steps which caused its slower performance. The battery efficiency directly depends on path length because the system uses a fixed 0.4% battery

consumption rate for each step. The A* and Hybrid systems maintained 84.4% battery life during their optimal path runs. The RL system produced paths of different lengths which led to efficiency levels between 82.8% and 84.4% across all environments. The Hybrid system achieved 83.2% battery efficiency in Environment 4 because it used 42 steps instead of the optimal 39 steps. The system achieved 84.4% battery efficiency in optimal trials but dropped to 83.2% when it used 42 steps. The path optimization approach achieved 100% success rate in all ten environments while experiencing only one collision which proved its highest level of reliability. The Hybrid system achieved 100% success rate in all ten environments while experiencing only one collision which proved its highest level of reliability. A* achieved perfect optimality and 100% goal success but it experienced nine dynamic collisions throughout ten trials which created safety risks in critical applications that need collision prevention.

Trade-off Analysis: The research data shows that the system must choose between achieving optimal results and maintaining flexibility and dependability. The A* algorithm generates perfect static plans but fails to adapt to changes which results in multiple collisions. The RL system achieves perfect adaptation through zero collisions during successful operations, yet its training process leads to suboptimal routes (41.4 steps on average) and complete system breakdowns (Environment 3). The Hybrid system produces results that approach optimal performance by achieving 90% optimal path success and 90% collision prevention and 100% mission completion with only 1.13% additional processing time. The system provides optimal performance for real-world applications because it maintains both reliability and safety standards.

Environmental Complexity Impact: The Hybrid system generated a 42-step path when dealing with Environment 4 which contained 69 static and 8 dynamic obstacles at its highest density level of density. All three algorithms successfully navigated through the environment by producing either optimal or near-optimal paths. The number of static obstacles in an environment determines how the system will make its local adaptation choices. The specific arrangement of obstacles in Environment 3 led to RL system failure but A* and Hybrid systems succeeded despite this environment. The A* algorithm experienced its highest number of collisions in Environments 9 and 10 because specific dynamic obstacle movement patterns made it difficult for non-adaptive algorithms to avoid collisions.

The catastrophic failure of RL in Environment 3 proves that Hybrid systems need to exist as a solution. The complete breakdown of RL during Environment 3 testing showed how untrained situations make this system fail completely. The Hybrid system's A* fallback mechanism proved essential because it successfully completed the same environment that RL failed to navigate. The system lacks safety features which make it unsuitable for dangerous operations although it shows excellent performance in trained environments. The Hybrid system achieved an 89% decrease in dynamic collisions when compared to A* which resulted in 1 collision versus 9 collisions throughout 10 trials. The system uses Q-Learning to adapt to moving obstacles which results in a significant decrease in dynamic collisions. The system achieves essential collision prevention which protects drones from damage and mission termination during real-world operations.

The Hybrid system achieved A* level path optimality in 90% of all tested environments. The Hybrid system produced paths that matched A*'s 39-step optimal solution in 9 out of 10 environments but Environment 4 required 42 steps which represented a 7.7% suboptimal result. The adaptation systems in the system do not create major path efficiency problems during most operational situations. The system operates at a minimal time cost because it adds only 71 milliseconds to the overall processing time. The system adds less than 1% to the total processing time which makes it insignificant for search and rescue operations that require extended periods of time.

All three algorithms produced perfect results when planning routes through static obstacles because they avoided any collisions during the 30 trial runs. The initial A* planning process successfully handled all known obstacles because it produced zero static collisions throughout the 30 trials. The system's ability to adapt successfully handles unpredictable situations but it does not fix planning-related problems. The system shows different weaknesses when operating in various environmental conditions. The RL system failed to handle Environment 3 because of its training limitations but the Hybrid system performed well in Environment 4 with its highest obstacle density and Environments 9-10 presented conditions that made even A*'s optimal planning unable to prevent multiple collisions. The selection of algorithms for deployment should consider the specific characteristics of the operating environment. The results indicate that the Hybrid system should be used for search and rescue drone operations because it provides both high reliability and strong safety features and near-optimal efficiency with minimal time overhead. The system delivers the

required performance profile for safety-critical missions because it achieves 100% success rate and 90% collision-free rate and 90% optimal path rate and 1.13% time increase.

5.2.3 Key Findings

The path optimization approach which combines A* with MR-Q-Learning achieves the research goal by optimizing global performance while adapting to changing conditions. The Hybrid approach shows better practicality than baseline methods because it achieves 90% optimal path rate and 90% collision-free rate and 100% success rate while adding only 1.13% time overhead. The testing environment automatically measured five performance metrics which Section 5.1 explained through code-level descriptions of step tracking and battery usage and collision detection and time efficiency and operational status. The Hybrid approach outperformed A* in all performance metrics according to Section 5.2 results which showed 90% optimal path success and 90% collision-free operation and 100% mission completion and 1.13% temporal overhead. The research confirmed RL's complete breakdown in Environment 3 which proved the necessity for backup systems and Hybrid outperformed A* by 89% in collision prevention and achieved near-optimal results in different obstacle arrangements. The experimental data confirms that uniting A* global planning with Q-Learning adaptation produces better results for path optimization and system flexibility and operational stability than using these methods separately. The experimental findings demonstrate that the hybrid system should operate in real-world drone search and rescue operations because it delivers high reliability and safety and operational efficiency.

Chapter 6 Conclusion and Future Work

6.1 Limitations and Constraints

While the current research demonstrates the effectiveness of the A*, MR-Q Learning, and Hybrid A* + MR-Q approaches for path planning in grid environments, several limitations must be acknowledged. One primary limitation lies in the replanning capability. In static environments, A* finds the optimal path efficiently, but it does not inherently handle dynamic obstacles. As a result, when obstacles move or appear unexpectedly, the precomputed A* path may become invalid, requiring a complete re-execution of the algorithm. This replanning process is computationally expensive and can lead to delays in real-time navigation scenarios. However, this limitation can be partially mitigated by MR-Q Learning, which allows local adaptive replanning based on learned Q-values. MR-Q Learning can dynamically adjust the agent's path when encountering unexpected obstacles without recalculating the entire path from scratch, making it better suited for environments with moderate levels of dynamism. Another limitation involves the scalability of the grid size. While A* and MR-Q Learning perform well on small to medium grids, their computational cost increases significantly with larger grids or higher obstacle densities. For very large environments, the time to compute optimal paths with A* can become prohibitive, and MR-Q Learning requires more episodes and memory to converge on effective policies. This limitation could impact practical applications where large-scale navigation is necessary. Finally, the random obstacle generation introduces variability that can affect reproducibility. While multiple maps were generated to mitigate bias, the stochastic nature of obstacle placement can influence path efficiency, steps taken, and battery consumption. Future research could explore standardized benchmarks or more controlled obstacle scenarios to provide consistent comparative evaluations. In summary, the key limitations include:

- (1) A*'s inefficiency in dynamic obstacle replanning, which can be improved with MR-Q Learning.
- (2) Computational and memory scalability concerns larger environments.
- (3) simplified battery and energy consumption modeling; and
- (4) stochastic variability in obstacle placement affects reproducibility.

Despite these limitations, the combination of MR-Q Learning with A* in approaches shows promise in addressing dynamic replanning challenges, making the framework more robust for real-world adaptive navigation.

6.2 Future Research Directions

Future work will concentrate on expanding the current framework to address these limitations while exploring advanced capabilities. The primary direction involves transitioning from 2D to 3D environment modeling to better symbolize urban environments with changing altitudes and complex structures. Implementing other highly developed obstacle movement patterns based on real disaster data would improve the simulation's realism and practical relevance. The integration of multi-drone coordination represents another significant avenue, enabling collaborative search patterns and resource sharing in large-scale rescue operations. Further development of the machine learning component will explore deep reinforcement approaches with neural network function approximators to handle larger state spaces and more complex environments. Incorporating real-world sensor data fusion, including thermal imaging and LIDAR simulations, would improve the system's practical applicability. Additional research will investigate predictive battery modeling that accounts for varying flight conditions and payload requirements, creating more accurate energy consumption forecasts for mission planning. The long-term vision includes developing a comprehensive drone autonomy framework that integrates path planning with real-time decision making, communication protocols, and mission management systems. Such advancements would significantly enhance the effectiveness of drone deployments in urban search and rescue missions, ultimately contributing to improved emergency response capabilities and increased survival rates in disaster scenarios.

6.3 Conclusion

The research accomplished its main objective to create intelligent real-time path planning system for autonomous drones operating in search and rescue missions with changing environmental conditions. The research required an integrated solution because traditional A algorithms fail in dynamic environments and pure reinforcement learning methods demonstrate poor sample efficiency and suboptimal convergence. The research has developed a new combined path planning system which unites the A* algorithm with Modified Reward Q-Learning (MR-QLearning) for enhanced performance. The main achievement of this research proves that uniting classical

methods with learning-based approaches produces better results than using either approach separately. The combined system uses A* algorithm to create an initial optimal path through static environmental knowledge while MR-QLearning performs real-time obstacle avoidance against dynamic obstacles using a shaped reward function and heuristic-based exploration. The system maintains mission continuity through its fallback mechanism to A* which activates when the learning agent encounters untrained situations. The proposed system underwent thorough testing through simulations conducted in a specially designed 2D grid-based environment. The experimental results showed that the hybrid system achieved high success rates in both static and dynamic environments by outperforming the individual A* and MR-QLearning algorithms. The combined system preserved A*'s optimal path performance in static environments and developed reinforcement learning ability to handle dynamic situations which resulted in better navigation and reduced path changes and lower battery usage. The research develops an operational drone navigation system which achieves optimal performance through adaptable control methods that maintain operational efficiency. The research proves that hybrid intelligence systems offer an excellent solution for operating autonomous agents in missions that require complex and unpredictable and time-sensitive operations like urban search and rescue. The research foundation established in this work enables future development into a deployable technology which will enhance emergency response team capabilities and save lives.

References

1. PAPYAN N, MELKONYAN A, KULHANDJIAN M, et al. AI-Based Drone Assisted Human Rescue in Disaster: Challenges and Opportunities[J]. arXiv preprint arXiv:2406.15875, 2024.
2. AL-DOSARI K, HUNAITI Z, BALACHANDRAN W. Systematic Review on Civilian Drones in Safety and Security Applications[J]. Drones, 2023, 7(3): 210.
3. DURGUN Y, DURGUN M. Smart environmental drone utilization for monitoring urban air quality[J]. Environmental Research and Technology, 2024, 7(2): 194-200.
4. ZHANG H. From Battlefield to Border: The Evolving Use of Drones in Surveillance Operations[J]. ITEJ: Information Technology Engineering Journals, 2024, 9(1): 44-52.
5. KOÇ M T. Drone Technologies and Applications[M]//Drones - Various Applications. London: IntechOpen, 2024.
6. Tullu A, Endale B, Wondosen A, Hwang H Y. Machine Learning Approach to Real-Time 3D Path Planning for Autonomous Navigation of Unmanned Aerial Vehicle[J]. Applied Sciences, 2021, 11(10): 4706.
7. L. Shan, R. Miura, T. Kagawa, F. Ono, H. -B. Li and F. Kojima, "Machine Learning-Based Field Data Analysis and Modeling for Drone Communications," in IEEE Access, vol. 7, pp. 79127-79135, 2019, doi: 10.1109/ACCESS.2019.2922544.
8. Sampath Boopathi, "Advancements in Machine Learning and AI for Intelligent Systems in Drone Applications for Smart City Developments", 2021, DOI: 10.4018/978-1-6684-9596-4.ch002
9. M. Z. Anwar, Z. Kaleem and A. Jamalipour, "Machine Learning Inspired Sound-Based Amateur Drone Detection for Public Safety Applications," in *IEEE Transactions on Vehicular Technology*, vol. 68, no. 3, pp. 2526-2534, March 2019, doi: 10.1109/TVT.2019.2893615.
10. Ramezani M, Amiri Atashgah M A. Energy-Aware Hierarchical Reinforcement Learning Based on the Predictive Energy Consumption Algorithm for Search and Rescue Aerial Robots in Unknown Environments[J]. Drones, 2024, 8(7): 283.

11. Zhang Z, Wu J, Dai J, He C. Optimal path planning with modified A-Star algorithm for stealth unmanned aerial vehicles in 3D network radar environment. *Proceedings of the Institution of Mechanical Engineers, Part G*. 2021;236(1):72-81. doi:10.1177/09544100211007381
12. AZAD H, MEHTA V, DADBOUD F, et al. Air-to-Air Simulated Drone Dataset for AI-powered problems[C]//2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC). IEEE, 2023: 1-7.
13. Humairah Hamid, G.R. Begh,, “Clustering based strategic 3D deployment and trajectory optimization of UAVs with A-star algorithm for enhanced disaster response,”*Physical Communication*, Volume 67,2024, 102536,, ISSN 1874-4907,https://doi.org/10.1016/j.phycom.2024.102536.
14. Zhang S, Li Y, Dong Q. Autonomous navigation of UAV in multi-obstacle environments based on a Deep Reinforcement Learning approach[J]. *Applied Soft Computing*, 2022, 115: 108194.
15. Li J, Liao C, Zhang W, Fu H, Fu S. UAV Path Planning Model Based on R5DOS Model Improved A-Star Algorithm. *Applied Sciences*. 2022; 12(22):11338. https://doi.org/10.3390/app122211338
16. KHOSRAVI M, ARORA R, ENAYATI S, et al. A search and detection autonomous drone system: from design to implementation[J]. *arXiv preprint arXiv:2211.15866*, 2022.
17. Mittal M, Mohan R, Burgard W, Valada A. Vision-Based Autonomous UAV Navigation and Landing for Urban Search and Rescue[M]//Asfour T, Mayer N, Šegvić S, Yao A. *Robotics Research*. ISRR 2019, SPAR 2020. Springer Proceedings in Advanced Robotics, vol 20. Springer, Cham, 2022: 575-592
18. Sandino J, Maire F, Caccetta P, et al. Drone-Based Autonomous Motion Planning System for Outdoor Environments under Object Detection Uncertainty[J]. *Remote Sensing*, 2021, 13(21): 4481.
19. HOHMANN N, BUJNY M, ADAMY J, et al. Multi-objective 3D path planning for UAVs in large-scale urban scenarios[C]//2022 IEEE Congress on Evolutionary Computation (CEC). IEEE, 2022: 1-8.
20. QIU S, DAI J, ZHAO D. Path planning of an unmanned aerial vehicle based on a multi-strategy improved pelican optimization algorithm[J]. *Biomimetics*, 2024, 9(10): 647.

21. AJITH V S, JOLLY K G. Unmanned aerial systems in search and rescue applications with their path planning: a review[J]. *Journal of Physics: Conference Series*, 2021, 2115(1): 012020.
22. KUMAR R, SINGH L, TIWARI R. Novel Reinforcement Learning Guided Enhanced Variable Weight Grey Wolf Optimization (RLV-GWO) Algorithm for Multi-UAV Path Planning[J]. *Wireless Personal Communications*, 2023, 131(3): 2093-2123.
23. J. Zhang, Y. Xian, X. Zhu and H. Deng, "A hybrid deep learning model for UAV path planning in dynamic environments," in *IEEE Access*, doi: 10.1109/ACCESS.2025.3557394.
24. Farshi, Esmaeil. (2022). Q*: A Hybrid Model Combining Q Learning and A Search Algorithms for Solving Complex Problems. 10.13140/RG.2.2.18291.50721.
25. Abdi, A.; Adhikari, D.; Park, J.H. A Novel Hybrid Path Planning Method Based on Q-Learning and Neural Network for Robot Arm. *Appl. Sci.* **2021**, *11*, 6770.
<https://doi.org/10.3390/app11156770>
26. R. Zanol, F. Chiariotti and A. Zanella, "Drone mapping through multi-agent reinforcement learning," 2019 IEEE Wireless Communications and Networking Conference (WCNC), Marrakesh, Morocco, 2019, pp. 1-7, doi: 10.1109/WCNC.2019.8885873.
27. Gharbi A (2024;), "A dynamic reward-enhanced Q-learning approach for efficient path planning and obstacle avoidance in mobile robotics". *Applied Computing and Informatics*, Vol. ahead-of-print No. ahead-of-print. <https://doi.org/10.1108/ACI-10-2023-0089>
28. E.S. Low, "A modified Q-learning path planning approach using distortion concept and optimization in dynamic environment for autonomous mobile robot" *Computers, Materials & Continua*, 2023
29. Xuan, D.T.; Hung, N.T.; Thang, V.T. A Comprehensive Review of Improved A* Path Planning Algorithms and Their Hybrid Integrations. *Automation* **2025**, *6*, 52.
30. Hee-Mun Park et al., "Hybrid path planning framework to integrate improved A*-DWA algorithms for enhancing path safety and efficiency", 2025.
31. Jayalakshmi K P et al., "Dynamic Spanning Tree Coverage with Q-Learning: A Hybrid Framework for Efficient Robot Navigation in Environments with Moving Obstacles," *IEEE Access*, 2025.
32. Yang Yang, Yuchao et al. , "Advancements in Q-learning meta-heuristic optimization algorithms: A survey", 2024

33. R. A. Arfianti, B. Lelono Widjiantoro, K. Indriawati and M. P. Shervind Maharani, "The Implementation of A* Algorithm for Global Path Planning in Autonomous Vehicles," *2023 International Conference on Advanced Mechatronics, Intelligent Manufacture and Industrial Automation (ICAMIMIA)*, Surabaya, Indonesia, 2023,
34. Wang, Y., Shen, B., Nan, Z., & Tao, W. (2024). An End-to-End Path Planner Combining Potential Field Method With Deep Reinforcement Learning. *IEEE Sensors Journal*, 24(16), 26584–26591. <https://doi.org/10.1109/jsen.2024.3425530>
35. Zhao, Y., & Gao, H. (2023). Dynamic Replanning for Autonomous Robots Using Hybrid A with Reinforcement Learning.* *IEEE Robotics and Automation Letters*, 8(1), 100–107. <https://doi.org/10.1109/LRA.2022.3215874>
36. Singh, A., & Saini, R. (2024). Comparative Analysis of A, D*, and Q-Learning for Real-Time Path Planning.* *IEEE Transactions on Artificial Intelligence*, 5(4), 231–242. <https://doi.org/10.1109/TAI.2024.3265890>